

FIAP ON

DIGITAL BUSINESS ENABLEMENT

SPRING MVC

6

LISTA DE FIGURAS

Figura 6.1 – Componentes da arquitetura MVC	7
Figura 6.2 – Criação de um projeto web com Maven – Parte 1.....	9
Figura 6.3 – Criação de um projeto web com Maven – Parte 2.....	10
Figura 6.4 – Criação de um projeto web com Maven – Parte 3.....	11
Figura 6.5 – Criação de um projeto web com Maven – Parte 4.....	12
Figura 6.6 – Ajuste da JDK configurado no projeto – Parte 1.....	13
Figura 6.7 – Ajuste da JDK configurado no projeto – Parte 2.....	14
Figura 6.8 – Ajuste da JDK configurado no projeto – Parte 3.....	14
Figura 6.9 – Configuração das dependências - pom.xml	15
Figura 6.10 – Configuração da aplicação web - web.xml	18
Figura 6.11 – Configuração do spring – app-context.xml – Parte 1.....	20
Figura 6.12 – Configuração do spring – app-context.xml – Parte 2.....	21
Figura 6.13 – Pasta para as views – Parte 1.....	25
Figura 6.14 – Pasta para as views – Parte 2.....	26
Figura 6.15 – Pasta para o driver do oracle – Parte 1.....	27
Figura 6.16 – Pasta para o driver do oracle – Parte 2.....	28
Figura 6.17 – Pasta para o driver do oracle – Parte 3.....	29
Figura 6.18 – Configuração do JPA – persistence.xml.....	30
Figura 6.19 – Primeiro Controller – Parte 1	31
Figura 6.20 – Primeiro Controller – Parte 2	32
Figura 6.21 – Diretório para as views – Parte 1	34
Figura 6.22 – Diretório para as views – Parte 2	35
Figura 6.23 – Nova página JSP – Parte 1	36
Figura 6.24 – Nova página JSP – Parte 2	37
Figura 6.25 – Executando o projeto – Parte 1	38
Figura 6.26 – Executando o projeto – Parte 2	39
Figura 6.27 – Executando o projeto – Parte 3	40
Figura 6.28 – Executando o projeto – Parte 4	41
Figura 6.29 – Página contato.jsp.....	42
Figura 6.30 – Teste da página contato.jsp	43
Figura 6.31 – Camada model da aplicação	49
Figura 6.32 – Criando o EstacionamentoController – Parte 1	50
Figura 6.33 – Criando o EstacionamentoController – Parte 2	51
Figura 6.34 – Criando a pasta para as páginas do estacionamento – Parte 1.....	52
Figura 6.35 – Criando a pasta para as páginas do estacionamento – Parte 2.....	53
Figura 6.36 – Criando a página de cadastro de estacionamento – Parte 1.....	53
Figura 6.37 – Criando a página de cadastro de estacionamento – Parte 2.....	54
Figura 6.38 – Teste da funcionalidade de cadastro de estacionamento – Parte 1....	57
Figura 6.39 – Teste da funcionalidade de cadastro de estacionamento – Parte 1....	58
Figura 6.40 – Teste da funcionalidade de cadastro de estacionamento – Parte 2....	60
Figura 6.41 – Teste da funcionalidade de cadastro de estacionamento – Parte 3....	61
Figura 6.42 – Criando a view para listar os estacionamentos – Parte 1.....	62
Figura 6.43 – Criando a view para listar os estacionamentos – Parte 2.....	62
Figura 6.44 – Teste da funcionalidade de listar estacionamentos.....	64
Figura 6.45 – Arquivos do bootstrap	65
Figura 6.46 – Pasta para o template	66
Figura 6.47 – Criando uma nova tag JSP - Parte 1.....	66

Figura 6.48 – Criando uma nova tag JSP - Parte 2.....	67
Figura 6.49 – Criando uma nova tag JSP - Parte 3.....	68
Figura 6.50 – Página inicial da aplicação	71
Figura 6.51 – Página de cadastro com o template	72
Figura 6.52 – Página de listagem com o template	74
Figura 6.53 – Botão de edição na listagem de estacionamentos	74
Figura 6.54 – Tela de atualização de estacionamento	78
Figura 6.55 – Mensagem de sucesso após a atualização.....	79
Figura 6.56 – Botão de exclusão na tela de listagem.....	81
Figura 6.57 – Modal para confirmação da exclusão do estacionamento.....	81
Figura 6.58 – Mensagem de sucesso após a exclusão.....	82

LISTA DE CÓDIGOS-FONTE

Código Fonte 6.1 – Configuração do pom.xml	17
Código Fonte 6.2 – Configuração do web.xml.....	19
Código Fonte 6.3 – Configuração do web.xml.....	22
Código Fonte 6.4 – Primeiro controller.....	33
Código Fonte 6.5 – Página index.jsp.....	37
Código Fonte 6.6 – Método contato no controller.....	42
Código Fonte 6.7 – Código da página contato.jsp.....	43
Código Fonte 6.8 – Entidade estacionamento.....	45
Código Fonte 6.9 – Interface do DAO Genérico.....	46
Código Fonte 6.10 – Classe do DAO Genérico.....	47
Código Fonte 6.11 – Interface EstacionamentoDAO.....	48
Código Fonte 6.12 – Classe EstacionamentoDAOImpl.....	48
Código Fonte 6.13 – Classe EstacionamentoController.....	52
Código Fonte 6.14 – Página JSP com o formulário de cadastro de estacionamento.....	55
Código Fonte 6.14 – Método POST de cadastro de estacionamento	58
Código Fonte 6.14 – Método de listagem do controller	61
Código Fonte 6.17 – Página JSP de listagem de estacionamentos	63

SUMÁRIO

6 SPRING MVC.....	6
6.1 Padrão MVC.....	6
6.2 Criando projeto Spring MVC.....	8
6.3 Primeiro Controller e View.....	30
6.4 Model.....	43
6.5 Adicionando o controller e a view – Cadastro e Listagem.....	50
6.6 Layout.....	64
6.7 Finalizando o CRUD! Atualizar e Excluir	74
REFERÊNCIAS.....	83

6 SPRING MVC

Spring é um framework de código aberto (*open source*), criado em 2002 por Rod Johnson. Esse framework foi criado para ser uma alternativa ao Java EE, no desenvolvimento de aplicações corporativas.

Assim como o Java EE, o *Spring* é composto por vários módulos com objetivos específicos, como persistência de dados, segurança, *web services*, aplicações web e o módulo original e principal: injeção de dependências.

O *Spring MVC* é um dos módulos mais populares do *Spring Framework* e um dos mais utilizados para desenvolvimento web na plataforma Java, isso se deve à facilidade de desenvolvimento de aplicações Java Web com a arquitetura MVC.

Antes de entrar de cabeça no framework *Spring MVC*, vamos falar sobre um dos padrões de projeto mais famosos e utilizados: o padrão MVC. Já que o *Spring MVC*, como o próprio nome diz, utiliza esse padrão como base para o desenvolvimento de aplicações.

6.1 Padrão MVC

MVC é um padrão arquitetural que divide uma aplicação em três camadas de componentes: *modelo*, *visão* e *controlador* (*Model*, *View* e *Controller*). Utilizado por muitas empresas e desenvolvedores, possui a intenção de estruturar melhor o código de grandes sistemas e determinar a responsabilidade de cada grupo de componentes. O padrão MVC pode ser utilizado para desenvolver os mais diversos tipos de aplicativos, desde desktop, até aplicações mobile e web. A figura “Componentes da arquitetura MVC” apresenta o diagrama dos componentes do padrão MVC e a suas relações.

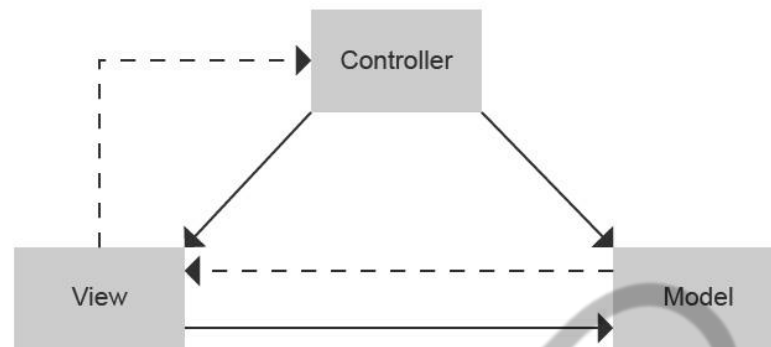


Figura 6.1 – Componentes da arquitetura MVC
Fonte: Google Imagens (2018)

Vamos detalhar a responsabilidade de cada componente:

- **Modelo (Model)** – São os componentes responsáveis pela lógica do negócio e pelo modelo de dados. Será responsável por validar, recuperar e armazenar o estado das informações em uma base de dados. O modelo também é usado para notificar sua *Visão (view)* para exibir as respostas atualizadas ao usuário do aplicativo.
- **Visão (View)** – Componentes de interface com o usuário. Tem a responsabilidade de exibir as informações atualizadas dos modelos. Pode usar tabelas para exibição de grandes conteúdos ou listas, ou formulários para digitação de dados que serão armazenados na base de dados.
- **Controlador (Controller)** – Responsável pelo fluxo da aplicação, gerencia as interações dos usuários, definindo quais modelos devem ser acionados e selecionando qual *visão (View)* será apresentada para o usuário.

A estrutura *MVC* tem como principais ideias a separação de conceitos, tornando cada componente responsável por um assunto e a reutilização de código. É um padrão utilizado em várias linguagens e plataformas de programação, dessa forma, possuem bibliotecas que facilitam a implementação do padrão *MVC*. Abaixo segue a lista dos frameworks mais conhecidos em suas respectivas linguagens:

- PHP – Cake PHP, Laravel e Symfony;

- Java – Spring MVC, VRaptor, Apache Struts;
- .NET – ASP.NET MVC;
- Python – Django, Zope;
- Ruby – Rails.

Com o uso do padrão *MVC*, podemos extrair algumas vantagens no desenvolvimento de sistemas corporativos, tais como:

- Facilidade de desenvolvimento de testes;
- Facilidade de gerenciamento da complexidade, devido à separação das camadas. Esse fator também ajuda na integração de grandes equipes de desenvolvedores;
- Controle completo do comportamento do aplicativo;
- Processamento centralizado das solicitações em um único controlador.

6.2 Criando projeto Spring MVC

Vamos criar um projeto com *Spring MVC* e *JPA* utilizando o Maven para o gerenciamento das dependências. Para isso, crie um Maven Project, acessando o menu **File > New > Maven Project**, conforme a figura 6.2.

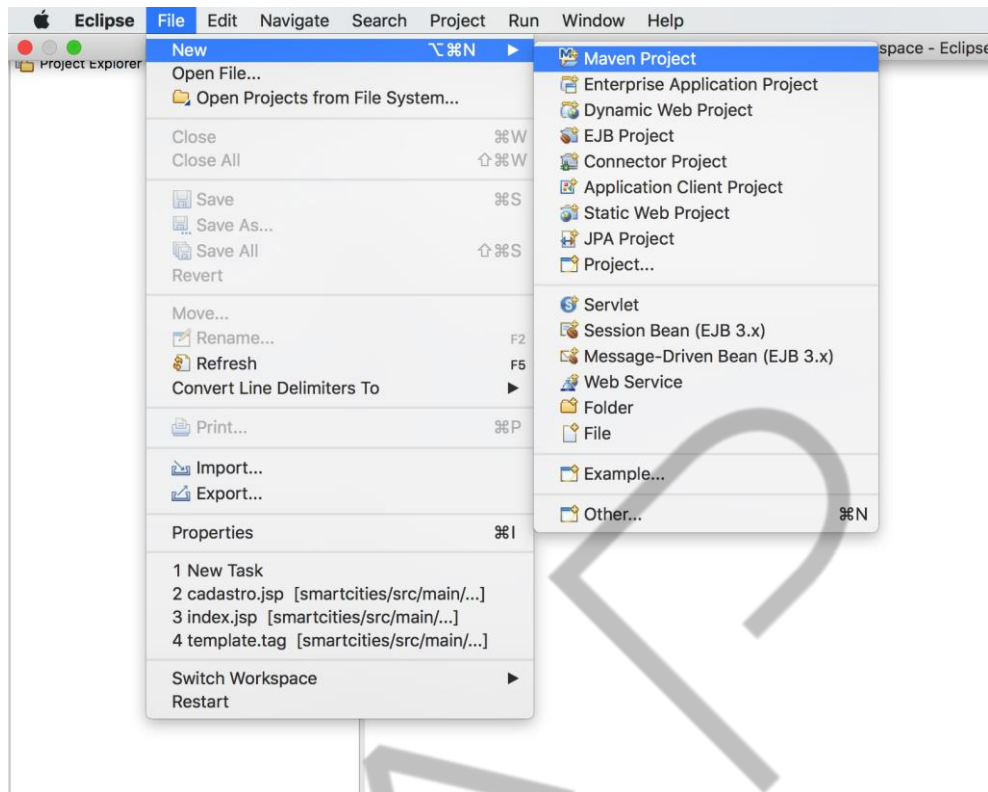


Figura 6.2 – Criação de um projeto web com Maven – Parte 1

Fonte: Elaborado pelo autor (2018)

No próximo passo, é possível escolher um diretório para salvar os arquivos do projeto. Vamos deixar marcada a opção “*Use default Workspace Location*” para gravar os arquivos no mesmo diretório da *workspace* em que o eclipse foi aberto (Figura Criação de um projeto *web* com Maven – Parte 2). Dessa forma, clique em Next.

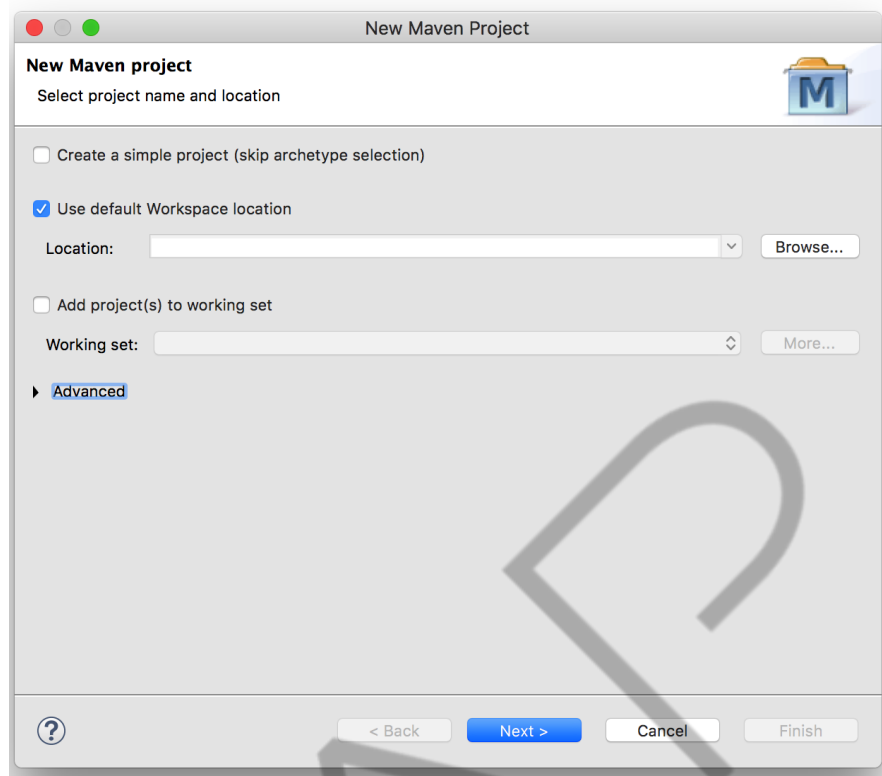


Figura 6.3 – Criação de um projeto web com Maven – Parte 2.
Fonte: Elaborado pelo autor (2018).

Agora é o momento de escolher o *archetype* do projeto, ou seja, a estrutura inicial que o projeto será criado. Escolha a opção “*maven-archetype-webapp*”, para criar um projeto *Web*. Depois clique em *Next* (Criação de um projeto web com Maven – Parte 3).

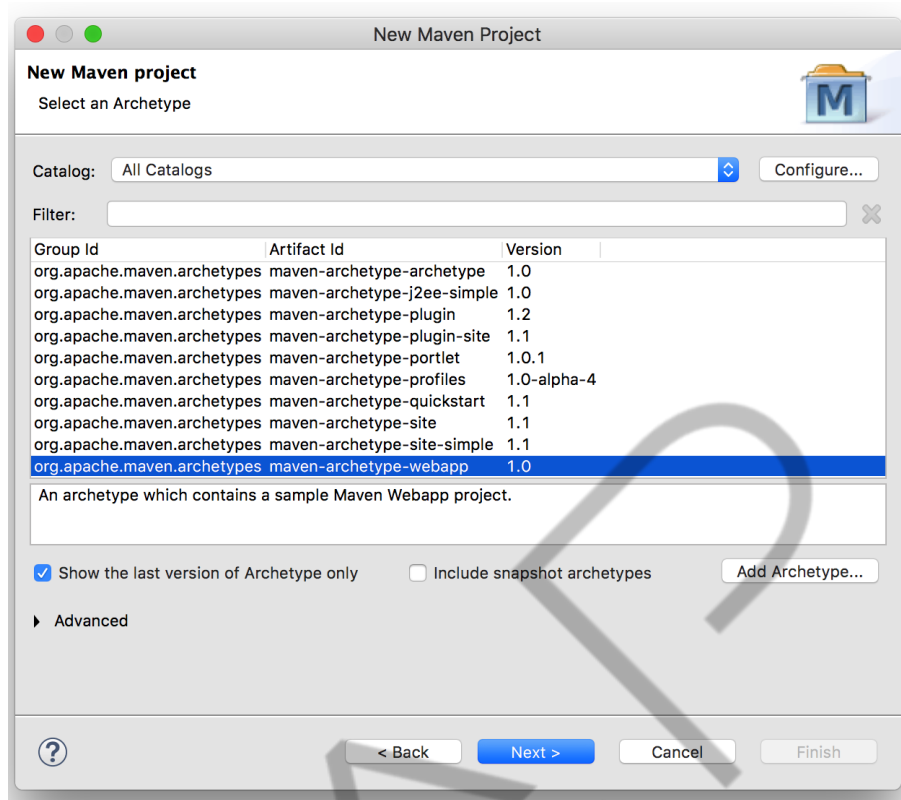


Figura 6.4 – Criação de um projeto web com Maven – Parte 3
Fonte: Elaborado pelo autor (2018)

O último passo é determinar o Group Id e Artifact Id, vamos configurar o Group Id como “*br.com.fiap*” e o Artifact Id como “*smartcities*”, que é o nome do nosso projeto.

A versão e o pacote podemos deixar conforme a configuração padrão (Criação de um projeto web com Maven – Parte 4).

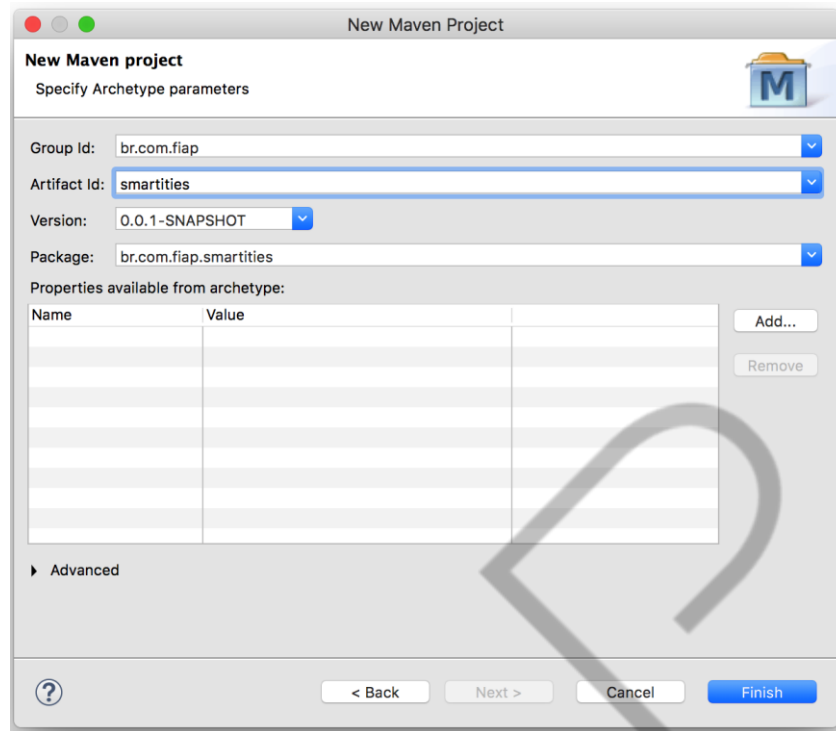


Figura 6.5 – Criação de um projeto web com Maven – Parte 4
Fonte: Elaborado pelo autor (2018)

Depois de criar o projeto, precisamos ajustar a versão da JRE, para utilizar o Java 8, em vez do Java 5. Para isso, clique com o botão direito do mouse em cima do projeto e escolha a opção “*Properties*”, conforme a (Ajuste da JDK configurado no projeto – Parte 1).

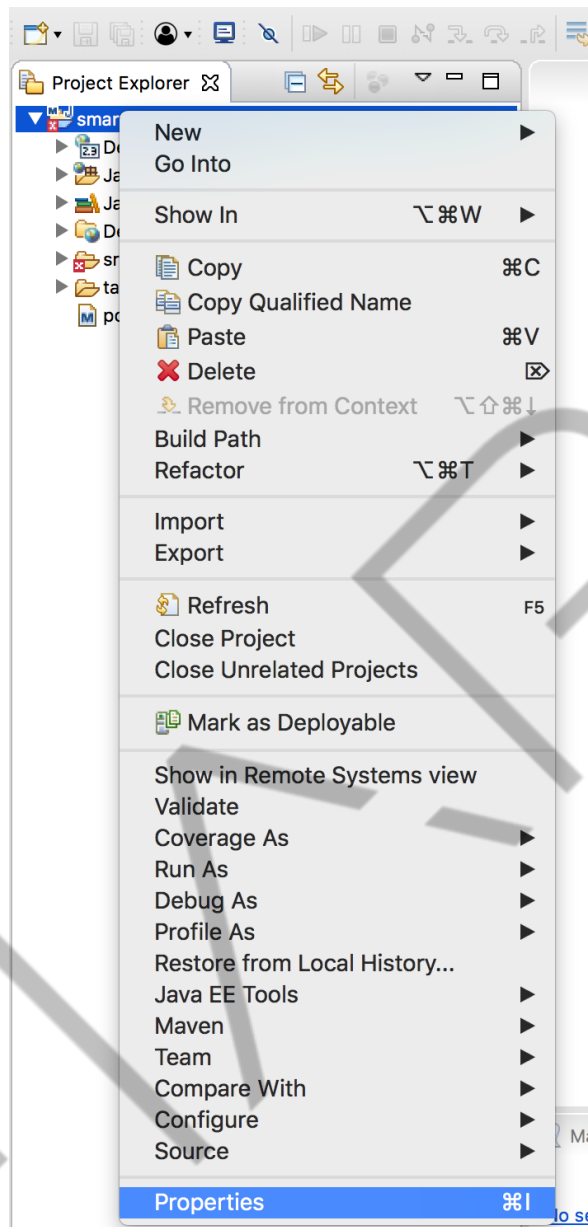


Figura 6.6 – Ajuste da JDK configurado no projeto – Parte 1

Fonte: Elaborado pelo autor (2018)

Do lado esquerdo, escolha “*Java Build Path*” e na aba de “*Libraries*” selecione a JRE e clique no botão “*Edit*” (Ajuste da JDK configurado no projeto – Parte 2).

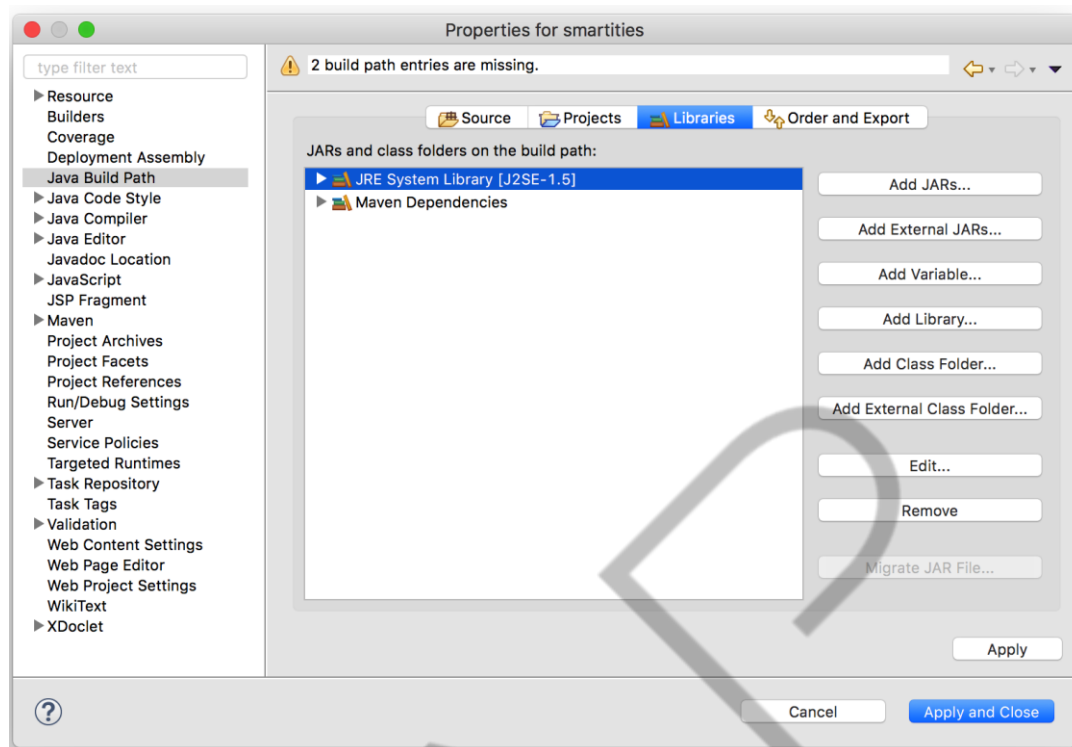


Figura 6.7 – Ajuste da JDK configurado no projeto – Parte 2
Fonte: Elaborado pelo autor (2018)

Por fim, escolha uma outra JRE, que pode ser a padrão do *workspace*.
“Workspace default JRE” (Ajuste da JDK configurado no projeto – Parte 3).

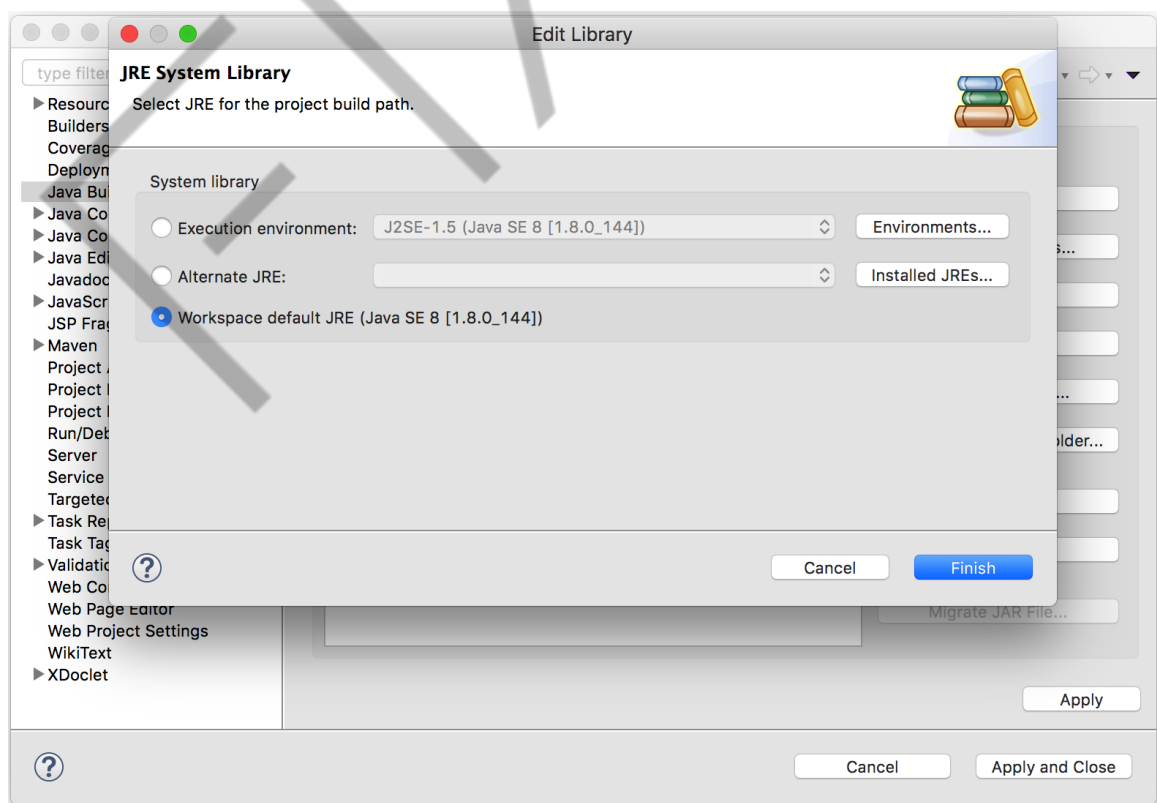


Figura 6.8 – Ajuste da JDK configurado no projeto – Parte 3
Fonte: Elaborado pelo autor (2018)

Para utilizar o *Spring* e o *JPA*, precisamos adicionar as bibliotecas. Para isso vamos utilizar o Maven editando o arquivo *pom.xml* (Configuração das dependências - *pom.xml*).

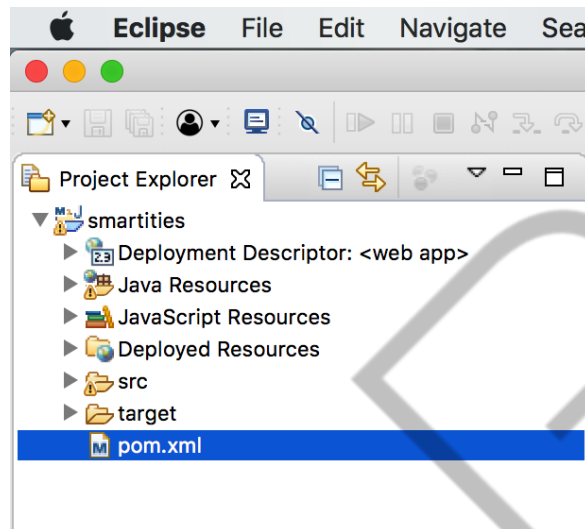


Figura 6.9 – Configuração das dependências - *pom.xml*
Fonte: Elaborado pelo autor (2018)

O projeto vem configurado com a dependência do *JUnit*, para testes unitários. Vamos adicionar as dependências do *Spring* e *JPA* logo abaixo da dependência do *JUnit*, conforme o código-fonte Configuração do *pom.xml*.

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/maven-v4_0_0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>br.com.fiap</groupId>
  <artifactId>smartities</artifactId>
  <packaging>war</packaging>
  <version>0.0.1-SNAPSHOT</version>
  <name>smartities Maven Webapp</name>
  <url>http://maven.apache.org</url>
  <dependencies>
    <dependency>
      <groupId>junit</groupId>
      <artifactId>junit</artifactId>
      <version>3.8.1</version>
      <scope>test</scope>
    </dependency>
    <dependency>
      <groupId>org.springframework</groupId>
      <artifactId>spring-aop</artifactId>
      <version>4.3.5.RELEASE</version>
    </dependency>
  </dependencies>
</project>
```

```

<dependency>
  <groupId>org.springframework</groupId>
  <artifactId>spring-webmvc</artifactId>
  <version>4.3.5.RELEASE</version>
</dependency>
<dependency>
  <groupId>org.springframework</groupId>
  <artifactId>spring-context</artifactId>
  <version>4.3.5.RELEASE</version>
</dependency>
<dependency>
  <groupId>org.springframework</groupId>
  <artifactId>spring-web</artifactId>
  <version>4.3.5.RELEASE</version>
</dependency>
<dependency>
  <groupId>commons-logging</groupId>
  <artifactId>commons-logging</artifactId>
  <version>1.2</version>
</dependency>
<dependency>
  <groupId>javax.servlet</groupId>
  <artifactId>jstl</artifactId>
  <version>1.2</version>
</dependency>
<!--
https://mvnrepository.com/artifact/javax.servlet.jsp/jsp-api
-->
<dependency>
  <groupId>javax.servlet.jsp</groupId>
  <artifactId>jsp-api</artifactId>
  <version>2.0</version>
  <scope>provided</scope>
</dependency>
<dependency>
  <groupId>javax.servlet</groupId>
  <artifactId>servlet-api</artifactId>
  <version>2.5</version>
</dependency>
<dependency>
  <groupId>org.hibernate</groupId>
  <artifactId>hibernate-
entitymanager</artifactId>
  <version>5.1.3.Final</version>
</dependency>
<dependency>
  <groupId>org.hibernate</groupId>
  <artifactId>hibernate-
validator</artifactId>
  <version>5.2.4.Final</version>
</dependency>

```



```
<dependency>
  <groupId>org.hibernate</groupId>
  <artifactId>hibernate-core</artifactId>
  <version>5.1.3.Final</version>
</dependency>
<dependency>

  <groupId>org.hibernate.javax.persistence</groupId>
  <artifactId>hibernate-jpa-2.1-
api</artifactId>
  <version>1.0.0.Final</version>
</dependency>
<dependency>
  <groupId>org.springframework</groupId>
  <artifactId>spring-orm</artifactId>
  <version>4.1.0.RELEASE</version>
</dependency>
</dependencies>
<build>
  <finalName>smartities</finalName>
</build>
</project>
```

Código Fonte 6.1 – Configuração do pom.xml
Fonte: Elaborado pelo autor (2018)

Salve o arquivo e aguarde o download das bibliotecas. Note que adicionamos as dependências do *spring mvc*, *jstl*, *servlet* e *hibernate*. A ideia é desenvolver uma aplicação *web* com *Spring MVC* e *JPA* com a implementação do Hibernate. Além disso, vamos utilizar os JSPs e JSTL para implementar as páginas web dinâmicas.

Além das bibliotecas, vamos precisar adicionar algumas configurações específicas do *Spring MVC*.

Abra o arquivo **web.xml** que está dentro dos diretórios “**src > main > webapp > WEB-INF**” (Figura - Configuração da aplicação *web* - *web.xml*).

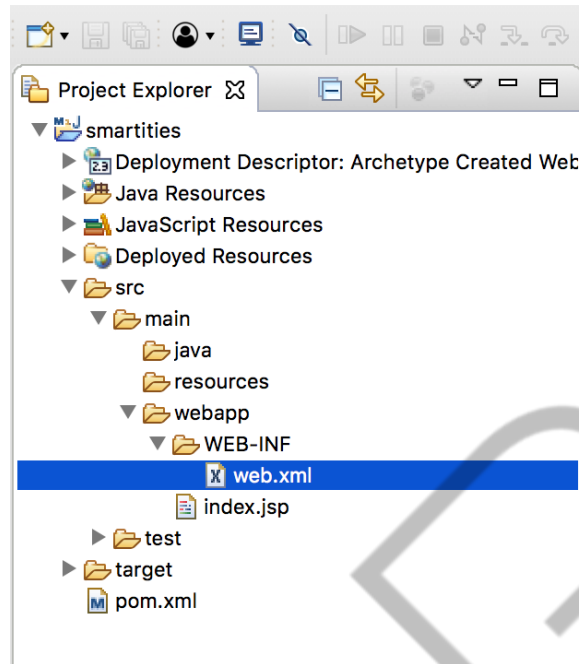


Figura 6.10 – Configuração da aplicação web - web.xml
 Fonte: Elaborado pelo autor (2018)

Neste arquivo configuramos o *servlet* do *spring* framework que será responsável por receber as requisições e redirecionar para as nossas classes *controller*. Para a *servlet*, será mapeada uma URL, neste caso, utilizamos somente a “/”, assim, todas as requisições serão tratadas por esta *servlet*. Também definimos o arquivo de configuração (*app-context.xml*) para o *spring mvc*, por meio da tag *<context-param>*. O código-fonte Configuração do *web.xml* apresenta a configuração final do arquivo *web.xml*.

```
<web-app>

    <listener>
        <listener-
class>org.springframework.web.context.ContextLoaderListener</
listener-class>
    </listener>

    <context-param>
        <param-name>contextConfigLocation</param-name>
        <param-value>/WEB-INF/app-context.xml</param-
value>
    </context-param>

    <servlet>
        <servlet-name>app</servlet-name>
        <servlet-
class>org.springframework.web.servlet.DispatcherServlet</serv
let-class>
```

```
<init-param>
  <param-name>contextConfigLocation</param-
name>
  <param-value></param-value>
</init-param>
<load-on-startup>1</load-on-startup>
</servlet>

<servlet-mapping>
  <servlet-name>app</servlet-name>
  <url-pattern>/</url-pattern>
</servlet-mapping>

</web-app>
```

Código Fonte 6.2 – Configuração do web.xml
Fonte: Elaborado pelo autor (2018)

O próximo passo é criar o arquivo “*app-context.xml*”. Para isso, clique com o botão direito do mouse na pasta “*WEB-INF*” e escolha a opção ‘*New > File*’, conforme a figura ” *Configuração do spring – app-context.xml – Parte 1*”.

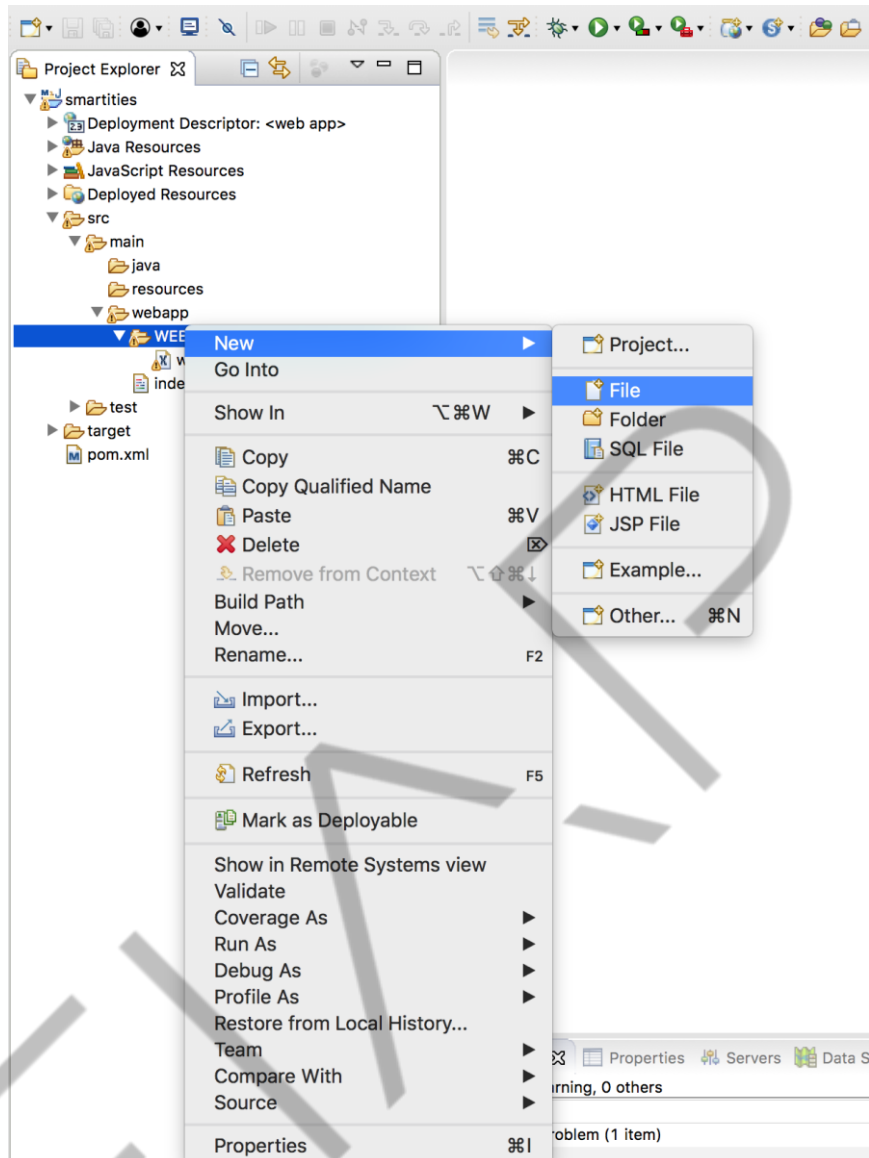


Figura 6.11 – Configuração do spring – app-context.xml – Parte 1
Fonte: Elaborado pelo autor (2018)

Dê o nome “*app-context.xml*”, certifique-se de que o nome esteja correto, pois a aplicação não irá funcionar, caso esteja diferente, e finalize o processo (Figura Configuração do *spring – app-context.xml – Parte 2*).

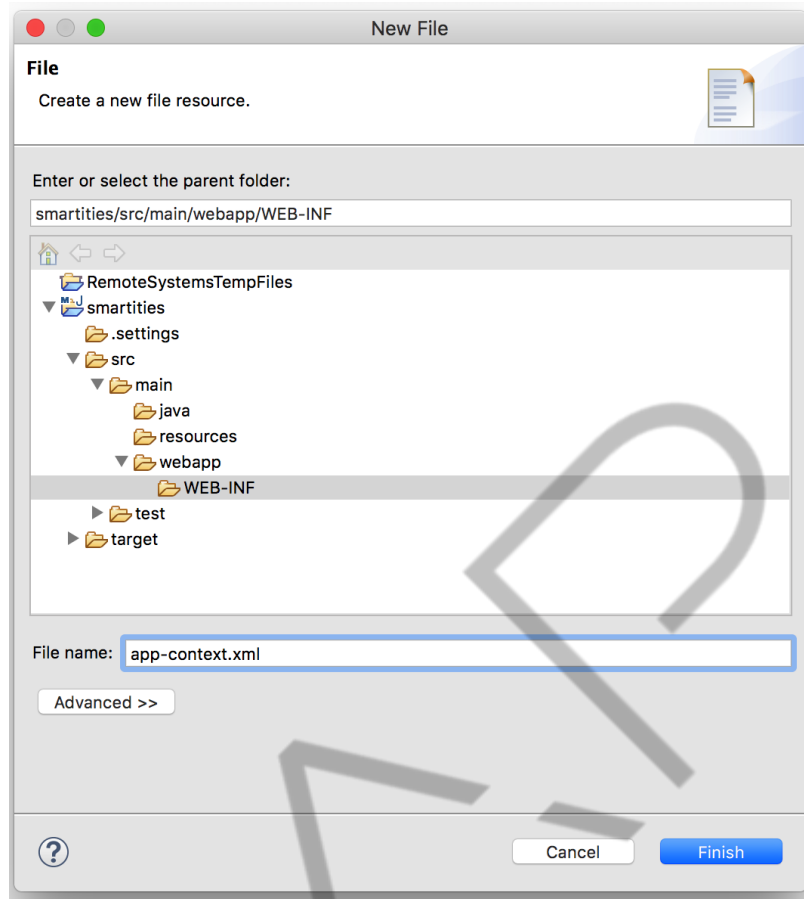


Figura 6.12 – Configuração do spring – app-context.xml – Parte 2
Fonte: Elaborado pelo autor (2018)

Neste arquivo precisamos realizar uma série de configurações para que o *framework Spring* funcione corretamente. Adicione as configurações conforme o código-fonte Configuração do *web.xml* para detalhar cada item a seguir.

```
<beans
xmlns="http://www.springframework.org/schema/beans"
xmlns:context="http://www.springframework.org/schem
a/context"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance"
xmlns:mvc="http://www.springframework.org/schema/mvc"
xmlns:tx="http://www.springframework.org/schema/tx"
xsi:schemaLocation="http://www.springframework.org/
schema/beans
http://www.springframework.org/schema/beans/spring-
beans-3.0.xsd http://www.springframework.org/schema/mvc
http://www.springframework.org/schema/mvc/spring-mvc-3.0.xsd
http://www.springframework.org/schema/context
http://www.springframework.org/schema/context/spring-context-
3.0.xsd http://www.springframework.org/schema/tx
http://www.springframework.org/schema/tx/spring-tx-4.1.xsd">
<context:component-scan base-
package="br.com.fiap.smartcities.controller" />
```

```

        <mvc:annotation-driven />
        <bean

            class="org.springframework.web.servlet.view.InternalResourceViewResolver">
                <property name="prefix" value="/WEB-INF/jsp/" />
            />

            <property name="suffix" value=".jsp" />
        </bean>
        <bean id="entityManagerFactory"

            class="org.springframework.orm.jpa.LocalContainerEntityManagerFactoryBean">
                <property name="persistenceUnitName" value="smartcities" />
                <property name="dataSource" ref="oracleDataSource" />
                <property name="jpaVendorAdapter">
                    <bean
                        class="org.springframework.orm.jpa.vendor.HibernateJpaVendorAdapter" />
                </property>
            </bean>
            <bean id="oracleDataSource"

                class="org.springframework.jdbc.datasource.DriverManagerDataSource">
                    <property name="driverClassName" value="oracle.jdbc.driver.OracleDriver" />
                    <property name="url" value="jdbc:oracle:thin:@oracle.fiap.com.br:1521:orcl" />
                    <property name="username" value="DESAFIO02" />
                    <property name="password" value="DESAFIO02" />
                </bean>

                <!-- Transações -->
                <bean id="transactionManager"
                    class="org.springframework.orm.jpa.JpaTransactionManager">
                    <property name="entityManagerFactory" ref="entityManagerFactory" />
                </bean>
                <tx:annotation-driven transaction-manager="transactionManager" />

            <mvc:default-servlet-handler/>

        </beans>

```

Código Fonte 6.3 – Configuração do web.xml
 Fonte: Elaborado pelo autor (2018)

A primeira configuração importante está na tag **<context:component-scan>**, que define o(s) pacote(s) onde os componentes do *Spring* devem estar. O *Spring* permite criar algumas classes específicas que serão responsáveis por alguma camada da aplicação, como os *controllers*, *services* e *repositories*. Veremos com mais detalhes cada um desses componentes no decorrer do capítulo. Agora o importante é configurar corretamente o lugar onde esses componentes estarão, para que o framework possa encontrá-los.

A tag **<mvc:annotation-driven />** habilita o uso de anotações para criar os componentes do *spring*. Dessa forma, não vamos precisar configurar os componentes por meio do arquivo *xml*, o que facilita e aumenta a produtividade do desenvolvedor.

As próximas configurações são as definições de alguns componentes prontos do *Spring* que serão responsáveis por gerenciar alguma parte da aplicação. Os componentes são configurados com a tag **<bean>** e uma classe. O primeiro componente configurado foi o **InternalResourceViewResolver**: **<bean class="org.springframework.web.servlet.view.InternalResourceViewResolver">**, que é responsável por gerenciar a camada *view* da aplicação. Nessa configuração determinamos qual será o tipo de arquivo (*jsp*) e onde esses arquivos devem estar localizados (*WEB-INF/jsp/*).

O segundo componente configurado é responsável por controlar a fábrica de *Entity Manager*, ou seja, não será preciso instanciar esse tipo de objeto, o *framework* se encarregará disso e muito mais! A configuração do *bean* foi realizada com a classe **org.springframework.orm.jpa.LocalContainerEntityManagerFactoryBean** e nela determinamos o nome da unidade de persistência (arquivo *persistence.xml* que iremos adicionar ao projeto) e o nome do *Data Source*, o componente que irá controlar as conexões com o banco de dados.

O próximo componente é exatamente o *Data Source*, nele configuramos o driver do banco de dados (oracle), a *url* para a conexão com o banco, o usuário e senha. Isso foi feito utilizando a classe **org.springframework.jdbc.datasource.DriverManagerDataSource**.

Outro ponto de que o *framework* irá se encarregar será o controle das transações, assim, configuramos um componente que se responsabilizará por isso:

org.springframework.orm.jpa.JpaTransactionManager, e também habilitamos o uso de anotações para “pedir” por uma transação ao *framework* quando for necessário.

A última configuração é a tag `<mvc:default-servlet-handler/>`, que permite que os arquivos de *css*, *js*, *imagens* e etc sejam recuperados sem passar pelo *framework*, já que são arquivos que simplesmente podem ser retornados para o *browser* sem nenhum tratamento.

Percebemos que o *Spring* vai nos ajudar na implementação da aplicação, não vamos precisar nos preocupar com a conexão com banco de dados, com o gerenciamento do *Entity Manager* e das transações. Esses são os objetivos dos *frameworks*, fazer o trabalho “braçal”, mecânico e deixar o desenvolvedor se preocupar com a inteligência da aplicação e as regras de negócio.

Acabamos de configurar a pasta onde devem estar os arquivos de *view* (*jsp*), assim, vamos criar essa pasta no local correto, conforme a figura “Pasta para as *views* – Parte 1”. Clique com o botão direito no diretório “*WEB-INF*” e escolha a opção “*New > Folder*”.

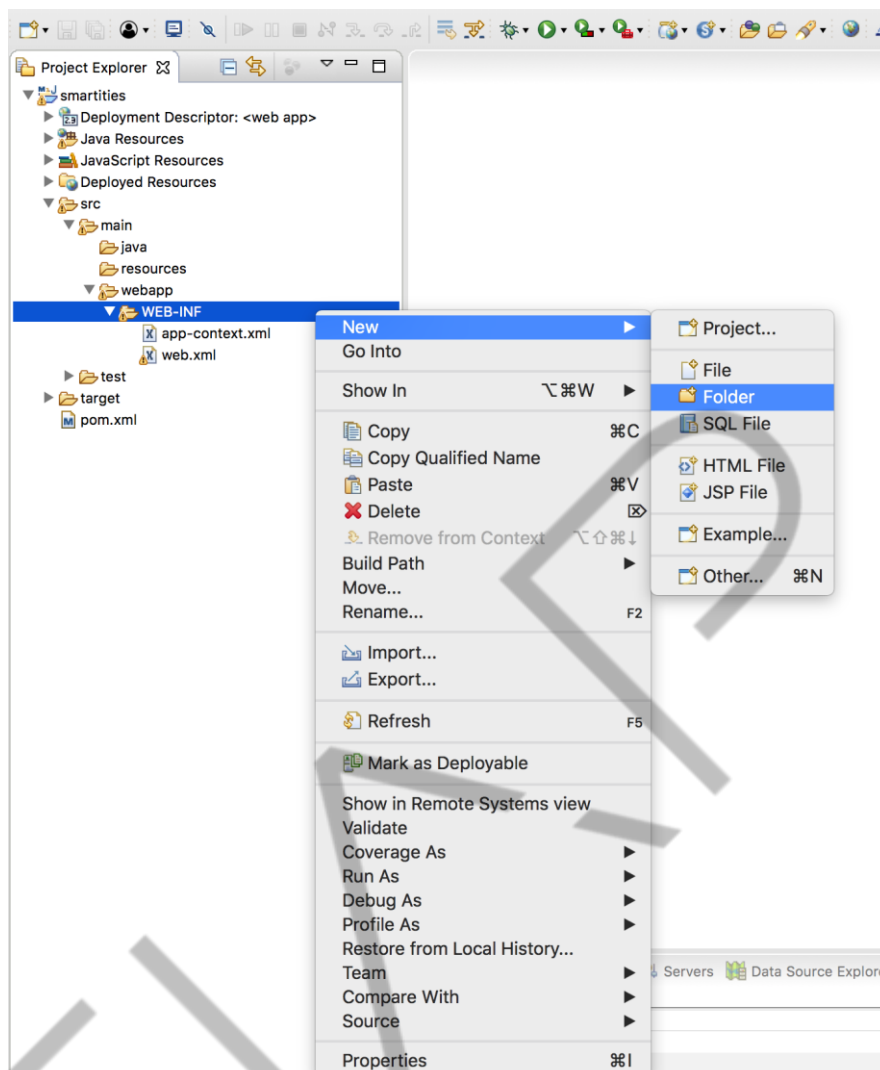


Figura 6.13 – Pasta para as views – Parte 1.

Fonte: Elaborado pelo autor (2018).

O diretório deve ter o nome que foi configurado anteriormente, neste caso **jsp** (Pasta para as views – Parte 2).

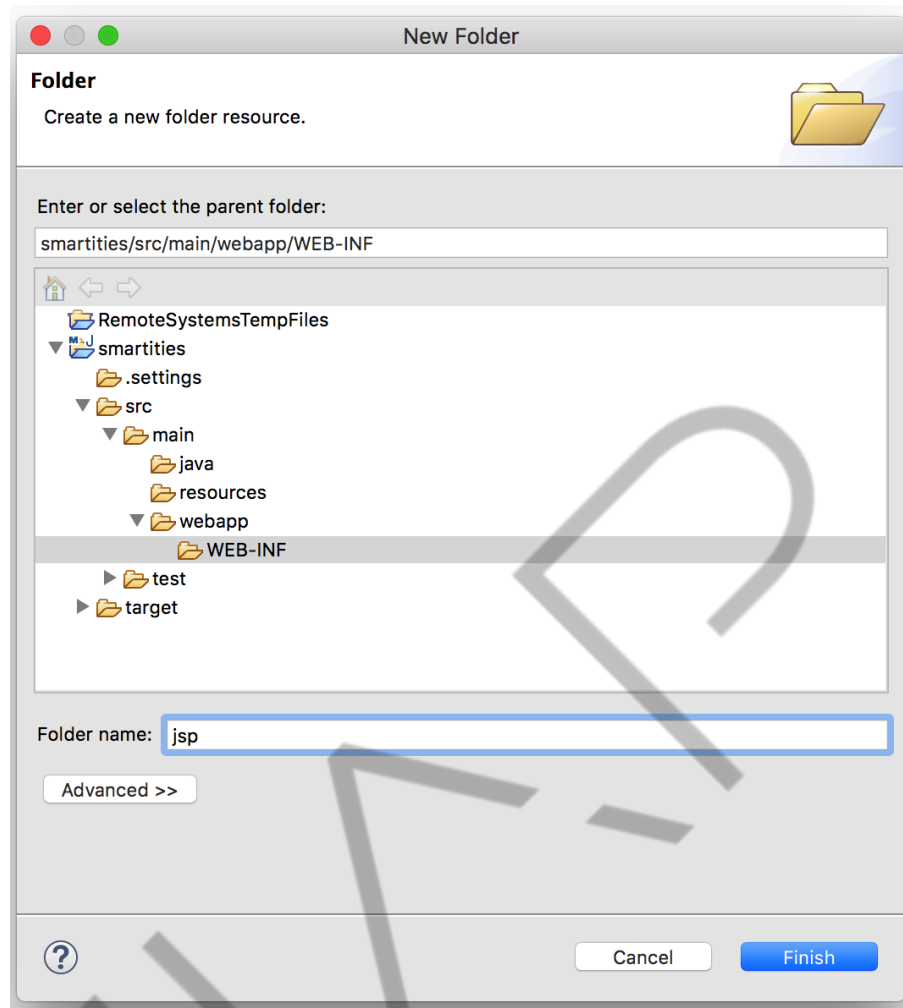


Figura 6.14 – Pasta para as views – Parte 2
Fonte: Elaborado pelo autor (2018)

Estamos quase lá! É preciso adicionar o driver do banco de dados, porém estamos utilizando o oracle. Por isso, vamos criar uma pasta chamada “*lib*” dentro do diretório “*WEB-INF*” para adicionar o driver. Essa pasta será identificada pelo servidor e será utilizada quando o *tomcat* for inicializado. Clique com o botão direito do mouse na pasta “*WEB-INF*” e escolha a opção “*New > Folder*”, Figura “Pasta para o driver do oracle – Parte 1”.

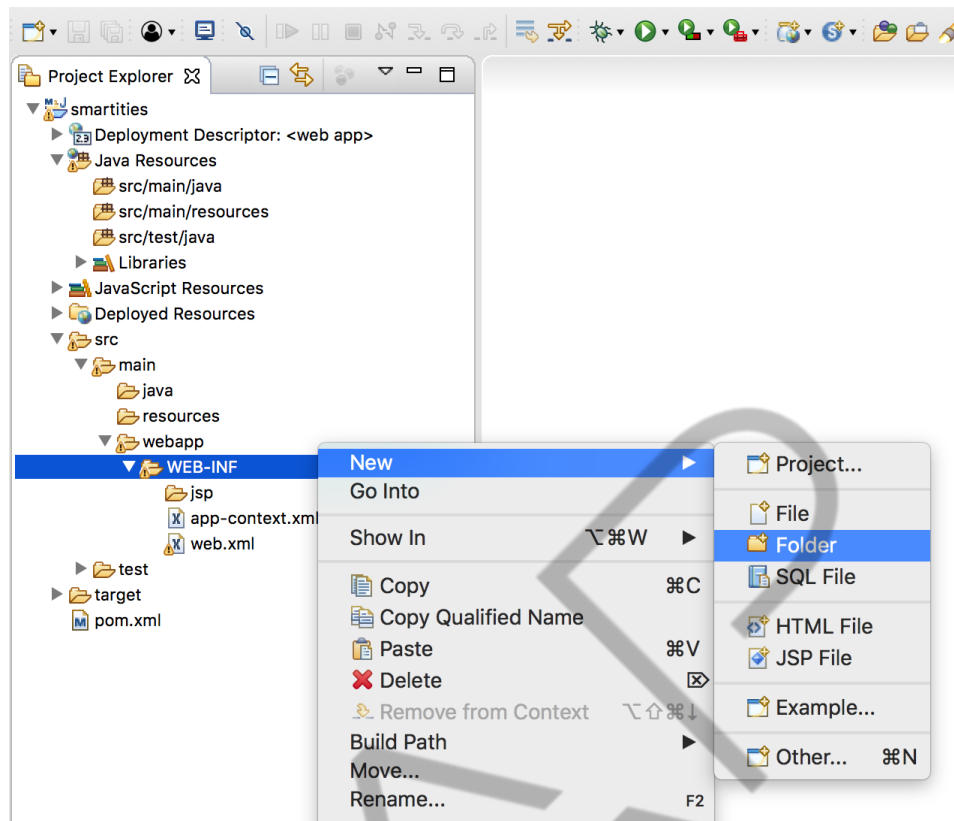


Figura 6.15 – Pasta para o driver do oracle – Parte 1
Fonte: Elaborado pelo autor (2018)

Dê o nome de “lib”, com todas as letras minúsculas e finalize o processo.
(Figura Pasta para o driver do oracle – Parte 2)

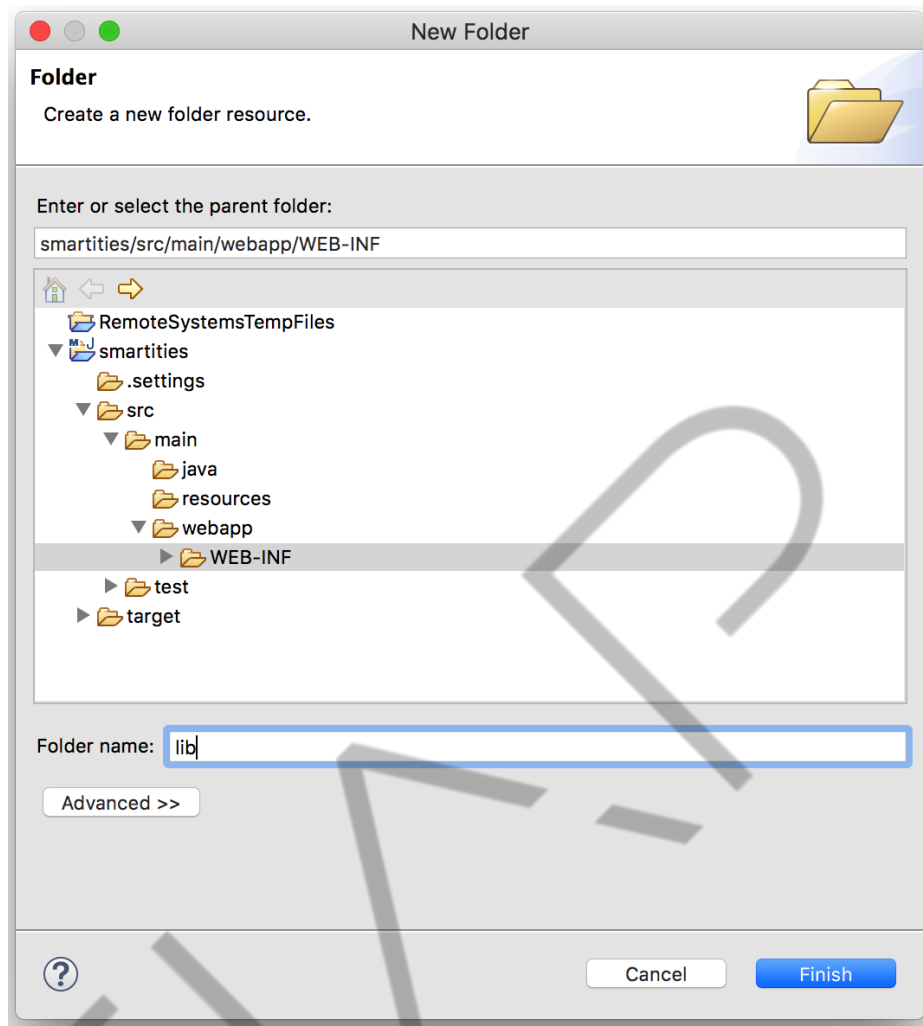


Figura 6.16 – Pasta para o driver do oracle – Parte 2
Fonte: Elaborado pelo autor (2018)

Por fim, copie e cole o driver do Oracle neste novo diretório (Figura Pasta para o driver do oracle – Parte 3).

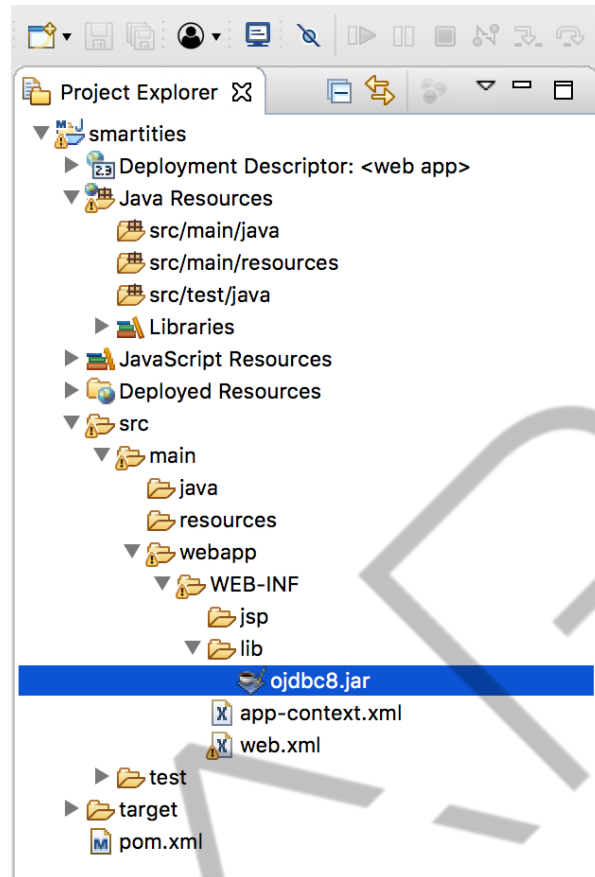


Figura 6.17 – Pasta para o driver do oracle – Parte 3
Fonte: Elaborado pelo autor (2018)

A última configuração necessária é o arquivo *persistence.xml* que deve estar dentro do diretório “META-INF” e este, por sua vez, deve estar dentro de Java Resources, *src/java/resources*, conforme a figura Configuração do JPA – *persistence.xml*.

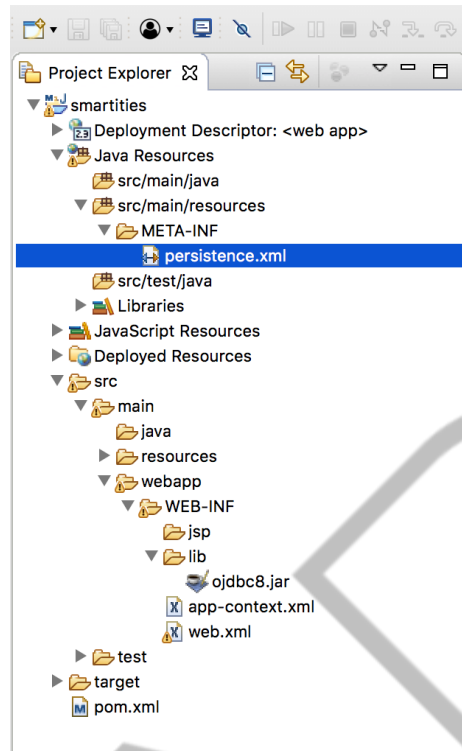


Figura 6.18 – Configuração do JPA – persistence.xml
Fonte: Elaborado pelo autor (2018)

Finalmente! Estamos com o projeto configurado e pronto para a implementação!

6.3 Primeiro Controller e View

Vamos implementar o primeiro *controller*. Relembrando, essa camada da aplicação é responsável por receber a requisição do usuário, invocar a camada *model* para recuperar informações e selecionar a *view* correta para apresentar os dados atualizados.

Para criar o primeiro controlador, clique com botão direito do mouse na pasta *src/main/java* e escolha a opção “New > Class” (Figura Primeiro Controller – Parte 1).

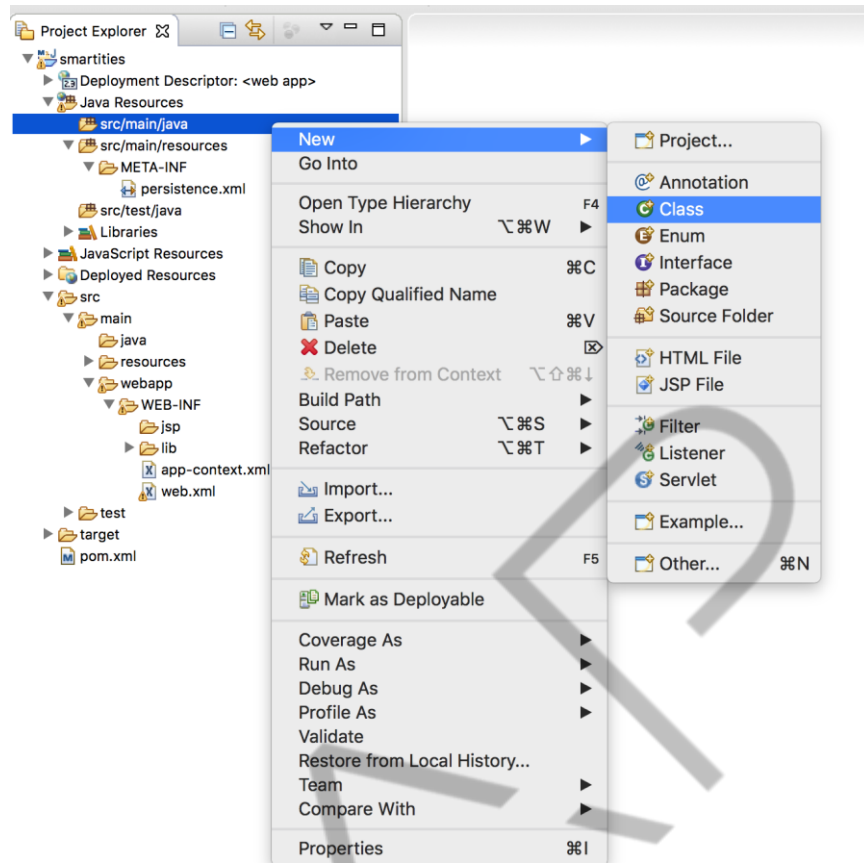


Figura 6.19 – Primeiro Controller – Parte 1
 Fonte: Elaborado pelo autor (2018)

Para o pacote da classe, utilize o mesmo pacote configurado no arquivo *app-context.xml*, para que o *framework* possa encontrá-lo. O pacote configurado foi **br.com.fiap.smartcities.controller**. A boa prática diz que os controladores devem terminar com a palavra **controller**. Dessa forma, podemos criar o **ClienteController**, **ProdutoController** e assim por diante. Para o nosso primeiro exemplo, vamos criar o **HomeController** (Figura Primeiro Controller – Parte 2).

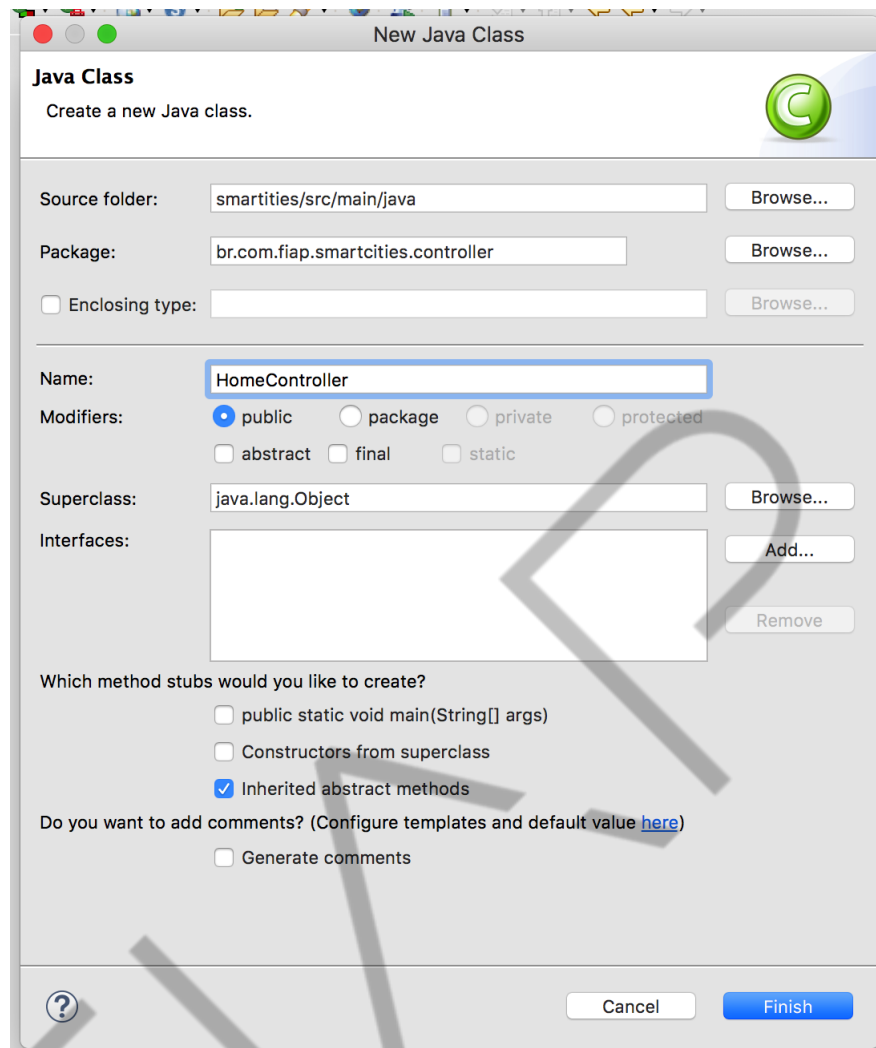


Figura 6.20 – Primeiro Controller – Parte 2
Fonte: Elaborado pelo autor (2018)

Para transformar uma classe em um componente do *spring* com a responsabilidade de *controller*, basta adicionar a anotação **@Controller**. Isso é possível, pois configuramos a utilização de anotações no arquivo *app-context.xml*.

Cada método dessa classe pode ser responsável por atender a uma requisição do usuário. Uma requisição deve ser enviada por um tipo de método do HTTP (GET ou POST) e uma URL. O método GET deve ser utilizado para realizar pesquisas e navegação na nossa aplicação. Já o método POST será utilizado para realizar cadastros, atualizações e remoções. A maior diferença entre eles é o lugar por onde são enviados os parâmetros da requisição. O GET envia os valores através da URL, por isso é possível ver na barra de endereço os dados enviados. Já o POST envia os parâmetros dentro do corpo da requisição, não alterando a URL.

Para mapear o método e o endereço, podemos utilizar as anotações **@GetMapping** e **@PostMapping**, que recebem a URL como parâmetro.

O código fonte *Primeiro controller* apresenta a implementação do *HomeController*:

```
package br.com.fiap.smartcities.controller;

import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.GetMapping;

@Controller
public class HomeController {

    @GetMapping("/")
    public String index() {
        return "home/index";
    }

}
```

Código Fonte 6.4 – Primeiro controller
Fonte: Elaborado pelo autor (2018)

Note que utilizamos as anotações **@Controller** e **@GetMapping("/")**. O primeiro foi para transformar a classe em um *controller*, o segundo foi para configurar o método para atender a requisição GET com a URL "/", ou seja, quando não informar nenhum caminho da URL (página inicial).

O retorno do método determina qual a página que será exibida para o usuário. Nesse caso, *"home/index"*. A palavra **home** determina o diretório onde deve estar a página, já o **index** é o nome do arquivo. Observe que não precisamos informar a extensão, pois ela pode variar. E configuramos anteriormente que os arquivos de *view* serão do tipo JSP.

Agora vamos criar o diretório **home** e a página **index** para completar o exemplo, figura Diretório para as *views* – Parte 1.

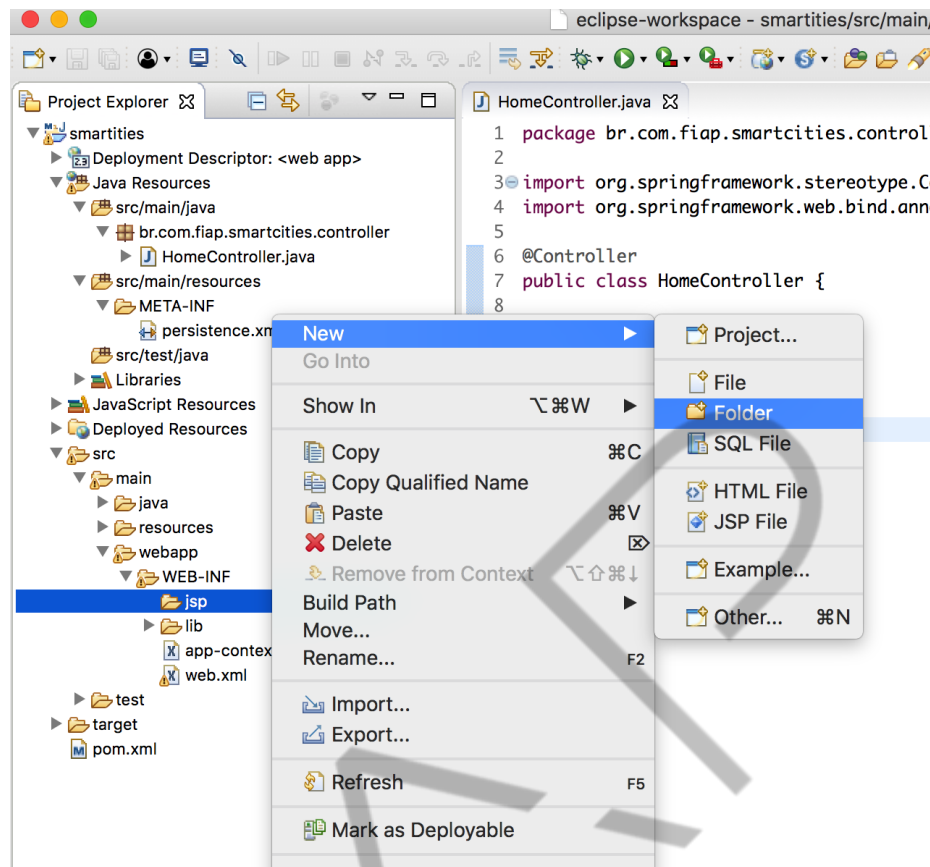


Figura 6.21 – Diretório para as views – Parte 1
Fonte: Elaborado pelo autor (2018)

Lembre-se que as views devem estar dentro do diretório “WEB-INF/jsp”, conforme configurado anteriormente.

Configure o nome da pasta como home (Figura Diretório para as views – Parte 2).

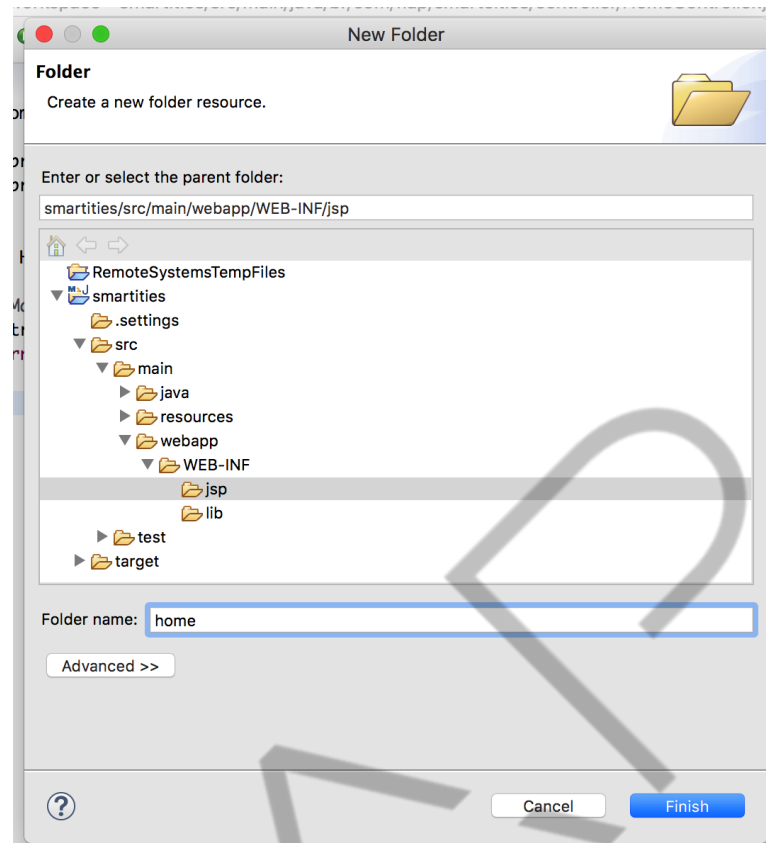


Figura 6.22 – Diretório para as views – Parte 2
Fonte: Elaborado pelo autor (2018)

Neste diretório crie uma nova página JSP, utilize o botão direito e escolha a opção “New > JSP File” (Figura Nova página JSP – Parte 1).

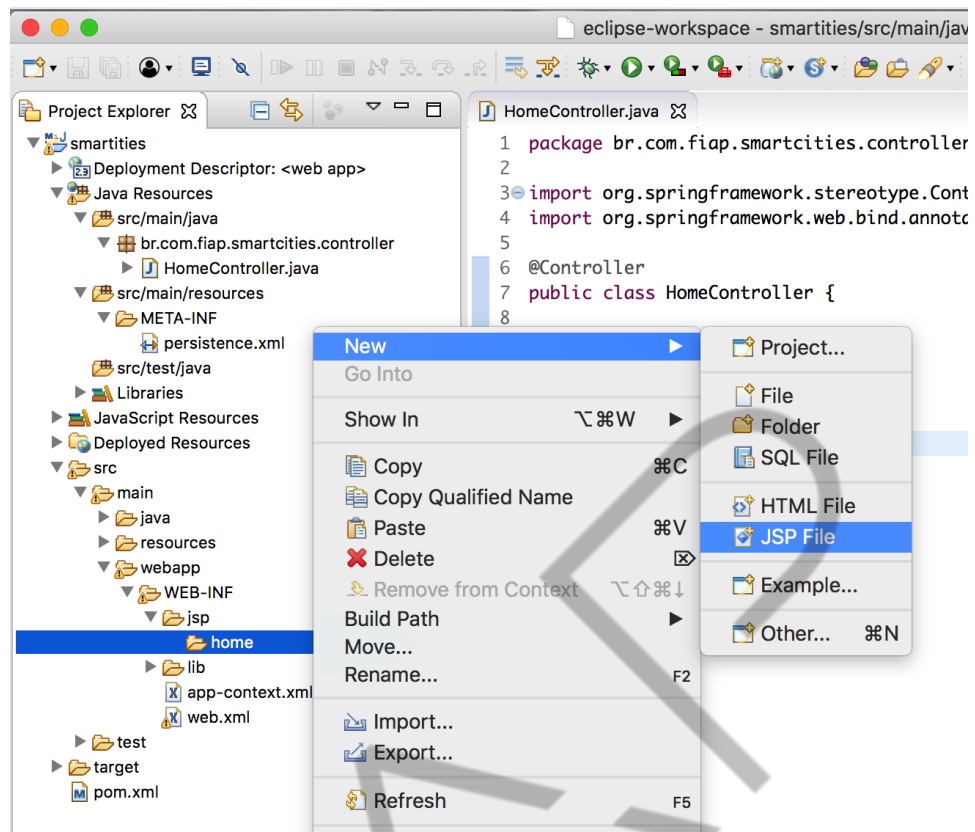


Figura 6.23 – Nova página JSP – Parte 1
Fonte: Elaborado pelo autor (2018)

O nome da página deve ser **index**, conforme a figura Nova página JSP – Parte

2.

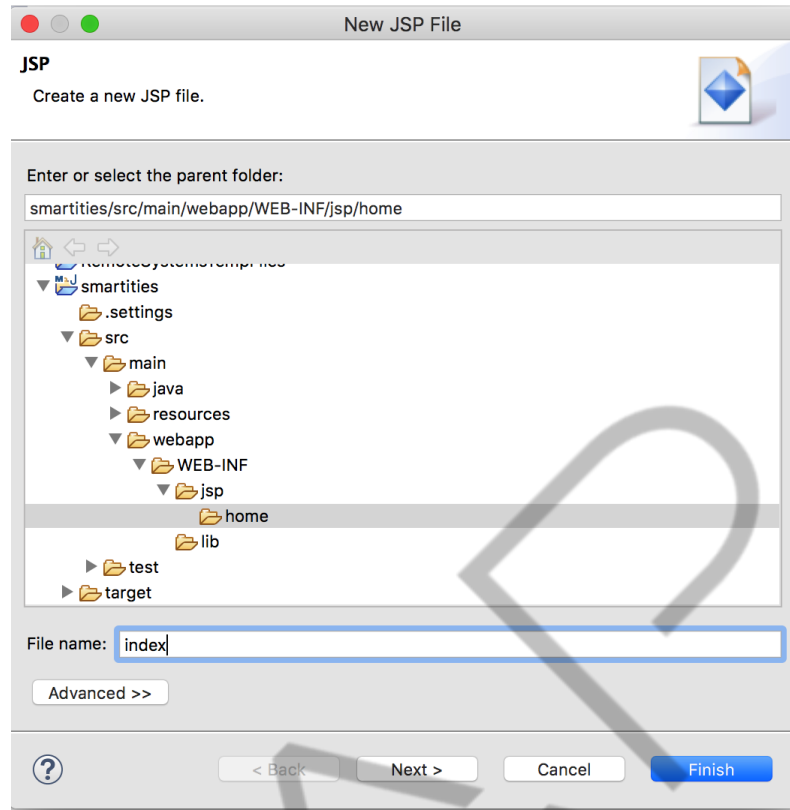


Figura 6.24 – Nova página JSP – Parte 2

Fonte: Elaborado pelo autor (2018)

Com isso criamos uma página JSP padrão, que aceita tags HTML e código Java. Vamos ajustar a página, adicionando um título para que possamos exibir para o usuário (*Código-fonte Página index.jsp*).

```
<%@ page language="java" contentType="text/html;
charset=UTF-8"
    pageEncoding="UTF-8"%>
<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01
Transitional//EN" "http://www.w3.org/TR/html4/loose.dtd">
<html>
<head>
<meta http-equiv="Content-Type" content="text/html;
charset=UTF-8">
<title>Olá Mundo!</title>
</head>
<body>

    <h1>Olá Mundo!</h1>

</body>
</html>
```

Código Fonte 6.5 – Página index.jsp

Fonte: Elaborado pelo autor (2018)

Vamos testar a aplicação!

Clique com o botão direito sobre o projeto e escolha “Run As > Run on Server”, conforme a Figura Executando o projeto – Parte 1.

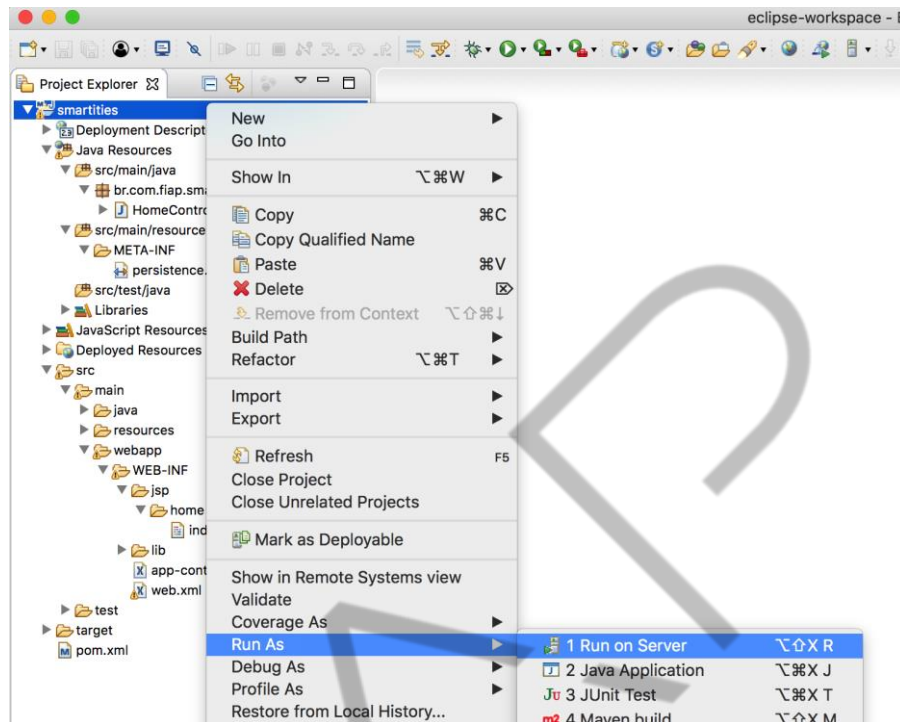


Figura 6.25 – Executando o projeto – Parte 1
Fonte: Elaborado pelo autor (2018)

Uma janela para a configuração do servidor será aberta. Neste momento, vamos utilizar o *Apache Tomcat 9* como servidor, dessa forma, escolha a opção correta e siga a configuração. (Figura Executando o projeto – Parte 2)

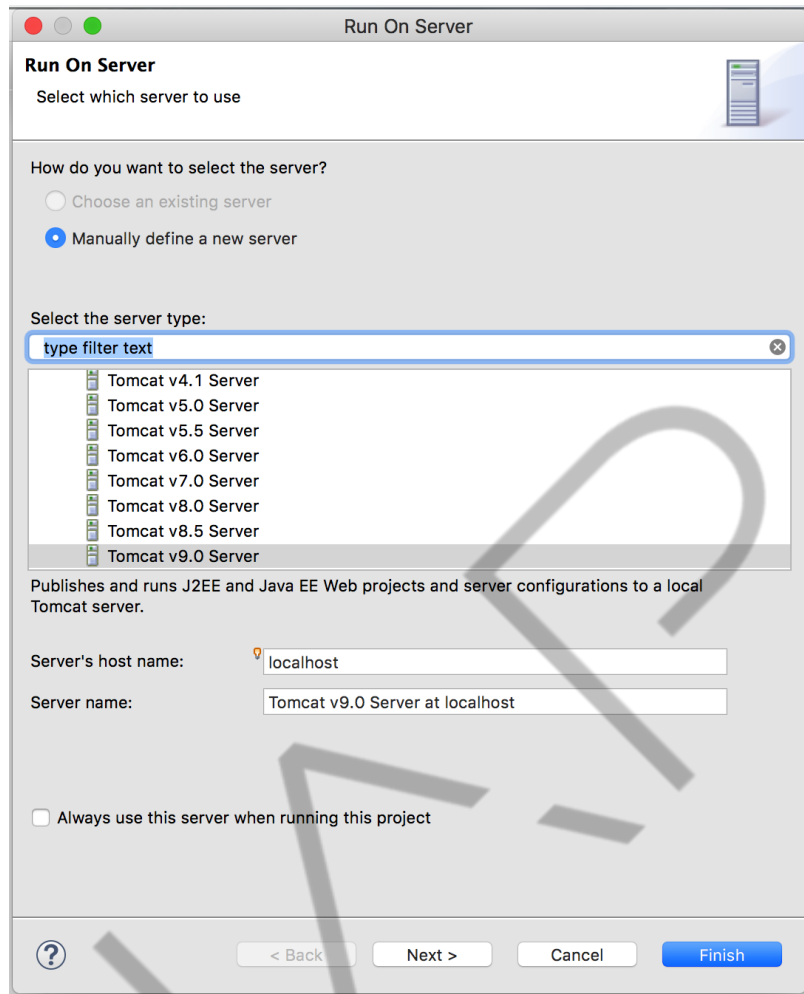


Figura 6.26 – Executando o projeto – Parte 2
Fonte: Elaborado pelo autor (2018)

O próximo passo é configurar onde o servidor *tomcat* está em sua máquina. Utilize o botão browser e navegue até o diretório do *tomcat* (Figura Executando o projeto – Parte 3). Esse servidor pode ser baixado do endereço <https://tomcat.apache.org/download-90.cgi>, não é necessária a instalação, basta descompactar o arquivo.

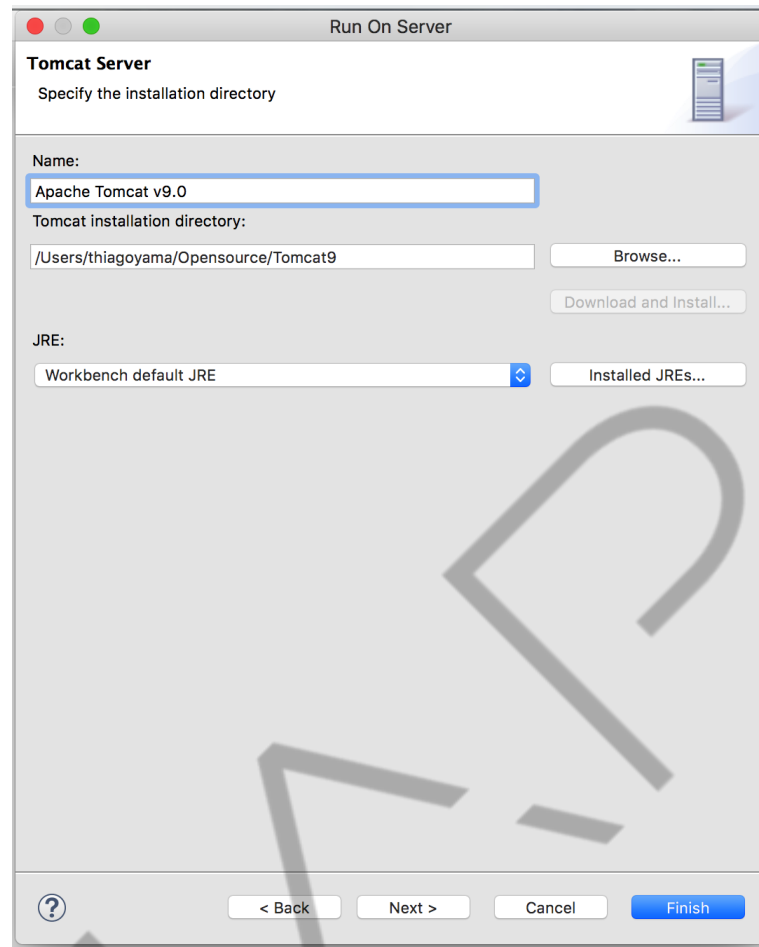


Figura 6.27 – Executando o projeto – Parte 3
Fonte: Elaborado pelo autor (2018)

Após isso, finalize o processo para que o *tomcat* possa inicializar. Após finalizar a inicialização uma página deve abrir automaticamente. Observe a URL, ela deve possuir o *host*, porta e o nome do projeto. Como configuramos o método para atender a requisição GET quando nenhum endereço específico for informado ("/"), o método vai atender a requisição e informar a página que será exibida para o usuário (*home/index*). O resultado deve ser igual à figura Executando o projeto – Parte 4.

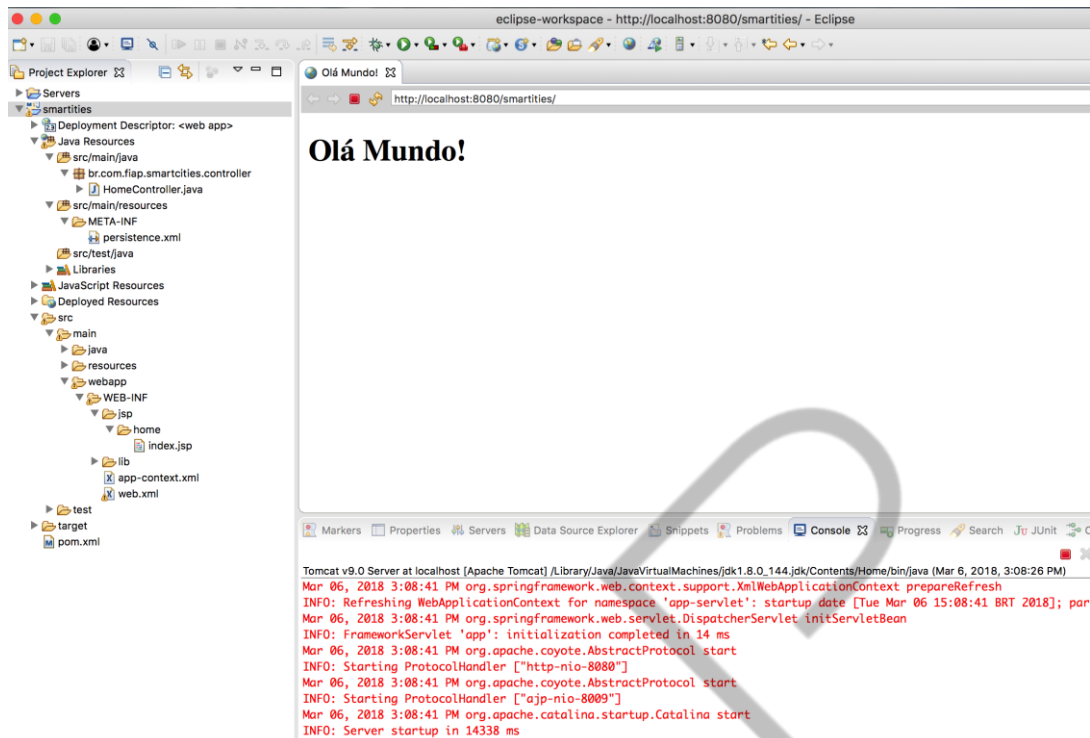


Figura 6.28 – Executando o projeto – Parte 4
Fonte: Elaborado pelo autor (2018)

Perfeito! Agora vamos adicionar mais uma página e um método na classe controladora, para entender melhor o sistema de rotas da aplicação. O método vai atender a URL “/contato” e vai exibir a view “home/contato”. Para isso, utilize a anotação `@GetMapping(“/contato”)` e faça o método retornar a *String* “home/contato”, conforme o código-fonte Método contato no *controller*.

```
package br.com.fiap.smartcities.controller;

import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.GetMapping;

@Controller
public class HomeController {

    @GetMapping("/")
    public String index() {
        return "home/index";
    }

    @GetMapping("/contato")
    public String contato() {
        return "home/contato";
    }
}
```

```
}
```

Código Fonte 6.6 – Método contato no controller
Fonte: Elaborado pelo autor (2018)

Agora é preciso criar a página “*contato.jsp*” no diretório “*jsp/home*” (Figura Página *contato.jsp*).

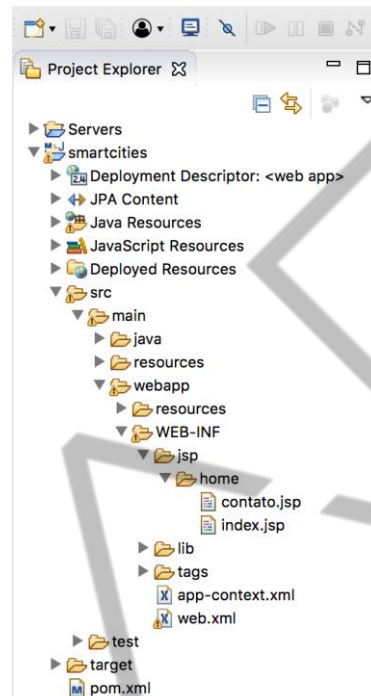


Figura 6.29 – Página *contato.jsp*
Fonte: Elaborado pelo autor (2018)

Ajuste a página, adicionando um título para que seja possível identificar a página contato. (Código Fonte Código da página *contato.jsp*)

```
<%@ page language="java" contentType="text/html;
charset=UTF-8"
    pageEncoding="UTF-8"%>
<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01
Transitional//EN" "http://www.w3.org/TR/html4/loose.dtd">
<html>
<head>
<meta http-equiv="Content-Type" content="text/html;
charset=UTF-8">
<title>Contato</title>
</head>
<body>

    <h1>Contato</h1>

</body>
```

```
</html>
```

Código Fonte 6.7 – Código da página contato.jsp
Fonte: Elaborado pelo autor (2018)

Agora vamos testar! Execute o projeto e ajuste a URL, conforme a figura Teste da página *contato.jsp*.

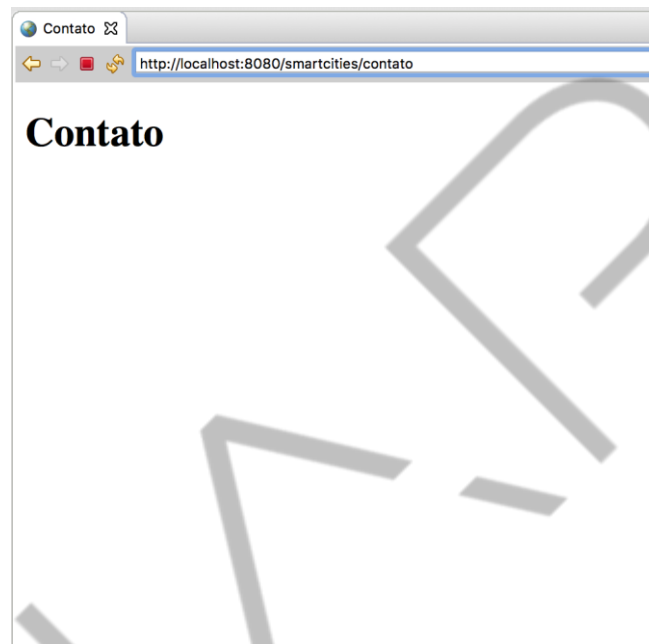


Figura 6.30 – Teste da página contato.jsp
Fonte: Elaborado pelo autor (2018)

Quando colocamos no final da URL o valor “**/contato**”, a requisição é direcionada para o *controller*, mais especificamente para o método **contato()** e ele, por sua vez, redireciona para a *view* “**home/contato**”, para a exibição da página como resposta.

6.4 Model

A nossa aplicação vai trabalhar com a gestão de estacionamento, permitindo que o usuário possa escolher o melhor local para estacionar de acordo com a quantidade de vagas disponíveis, preço e localização.

Dessa forma, vamos codificar o *backend* do sistema. Iremos utilizar o *JPA/Hibernate* para acessar o banco de dados Oracle. Toda a configuração já foi realizada junto da criação do projeto.

Primeiro, vamos desenvolver a Entidade, a classe que representa a tabela T_ESTACIONAMENTO no banco de dados. A tabela deve possuir as colunas para armazenar o código, nome, endereço, quantidade de vagas e preço.

Crie a classe **Estacionamento** no pacote **br.com.fiap.smartcities.entity** e implemente a classe, conforme o exemplo: "Entidade estacionamento".

```
package br.com.fiap.smartcities.entity;

import javax.persistence.Column;
import javax.persistence.Entity;
import javax.persistence.GeneratedValue;
import javax.persistence.GenerationType;
import javax.persistence.Id;
import javax.persistence.SequenceGenerator;
import javax.persistence.Table;

@Entity
@Table(name="T_ESTACIONAMENTO")
public class Estacionamento {

    @Id
    @Column(name="cd_estacionamento")
    @GeneratedValue(strategy=GenerationType.SEQUENCE, generator="estacionamento")
    @SequenceGenerator(name="estacionamento", sequenceName="SQ_T_ESTACIONAMENTO", allocationSize=1)
    private int codigo;

    @Column(name="nm_estacionamento", nullable=false, length=100)
    private String nome;

    @Column(name="ds_endereco", nullable=false)
    private String endereco;

    @Column(name="nr_vaga")
    private int vagas;

    @Column(name="vl_estacionamento")
    private float preco;

    public int getCodigo() {
        return codigo;
    }

    public void setCodigo(int codigo) {
        this.codigo = codigo;
    }
}
```

```
    public String getNome() {  
        return nome;  
    }  
  
    public void setNome(String nome) {  
        this.nome = nome;  
    }  
  
    public String getEndereco() {  
        return endereco;  
    }  
  
    public void setEndereco(String endereco) {  
        this.endereco = endereco;  
    }  
  
    public int getVagas() {  
        return vagas;  
    }  
  
    public void setVagas(int vagas) {  
        this.vagas = vagas;  
    }  
  
    public float getPreco() {  
        return preco;  
    }  
  
    public void setPreco(float preco) {  
        this.preco = preco;  
    }  
}
```

Código Fonte 6.8 – Entidade estacionamento
Fonte: Elaborado pelo autor (2018)

Continuando, é preciso criar a camada de acesso aos dados, o famoso DAO. Para isso, podemos implementar o DAO genérico que irá possuir as funcionalidades básicas (CRUD) para todas as entidades, o que irá facilitar quando adicionarmos mais entidades na aplicação.

O DAO deve ser formado por uma interface e uma classe. Primeiro crie a interface e declare as assinaturas dos métodos que todo DAO deve possuir. Não é preciso implementar o corpo do método na interface. Para deixar o DAO genérico, vamos utilizar o *generics* do Java, declarando dois argumentos que depois serão substituídos pela classe da entidade e a classe que mapeia a chave primária da tabela.

A *interface* do DAO é apresentada no código 6.9. A *interface* se chama **GenericDAO** e foi implementada dentro do pacote **br.com.fiap.smartcities.dao**.

```
package br.com.fiap.smartcities.dao;

import java.util.List;

public interface GenericDAO<T,K> {

    void cadastrar(T entidade);

    void atualizar(T entidade);

    T buscar(K chave) ;

    void remover(K chave) throws Exception ;

    List<T> listar();

}
```

Código Fonte 6.9 – Interface do DAO Genérico.
Fonte: Elaborado pelo autor (2018).

As letras T e K serão substituídas pela classe da entidade e chave primária posteriormente. Talvez você esteja sentindo falta do método **commit()**, esse método não será mais necessário, pois o *Spring Framework* irá gerenciar as transações, isso foi configurado junto da criação do projeto.

Depois da *interface*, precisamos da classe que irá implementar a *interface*. Crie a classe **GenericDAOImpl** dentro do pacote **br.com.fiap.smartcities.dao.impl**.

A classe deve implementar todos os métodos da *interface* (Código Classe do DAO Genérico).

```
package br.com.fiap.smartcities.dao.impl;

import java.lang.reflect.ParameterizedType;
import java.util.List;
import javax.persistence.EntityManager;
import javax.persistence.PersistenceContext;
import br.com.fiap.smartcities.dao.GenericDAO;

public class GenericDAOImpl<T,K> implements GenericDAO<T,
K>{

    @PersistenceContext
```

```

        protected EntityManager em;

        private Class<T> clazz;

        @SuppressWarnings("unchecked")
        public GenericDAOImpl() {
            clazz = (Class<T>) ((ParameterizedType)
getClass()

        .getGenericSuperclass()).getActualTypeArguments()[0];
        }

        public void cadastrar(T entidade) {
            em.persist(entidade);
        }

        public void atualizar(T entidade) {
            em.merge(entidade);
        }

        public T buscar(K chave) {
            return em.find(clazz, chave);
        }

        public void remover(K chave) throws Exception {
            T entidade = buscar(chave);
            if (entidade == null) {
                throw new Exception("Entidade não
encontrada");
            }
            em.remove(entidade);
        }

        public List<T> listar() {
            return em.createQuery("from " +
clazz.getName(), clazz).getResultList();
        }
    }

```

Código Fonte 6.10 – Classe do DAO Genérico

Fonte: Elaborado pelo autor (2018)

Outro detalhe importante é a anotação **@PersistenceContext** que está sobre o atributo do tipo **EntityManager**. Essa anotação faz com que o *Spring Framework* instancie um objeto do tipo **EntityManager** e insira nesse atributo, ou seja, é realiza a famosa injeção de dependência. Dessa forma, não precisamos instanciar os **EntityManager** e muito menos o **EntityManagerFactory**, todos esses objetos serão gerenciados pelo *framework*.

O próximo passo é criar o DAO para o estacionamento, utilizando a herança para obter os métodos básicos do DAO genérico.

Crie no mesmo pacote do GenericDAO a *interface EstacionamentoDAO* e estenda a interface genérica (*Código Interface EstacionamentoDAO*).

```
package br.com.fiap.smartcities.dao;

import br.com.fiap.smartcities.entity.Estacionamento;

public interface EstacionamentoDAO extends
GenericDAO<Estacionamento, Integer>{

}
```

Código Fonte 6.11 – Interface EstacionamentoDAO.
Fonte: Elaborado pelo autor (2018).

Agora vamos desenvolver o **EstacionamentoDAOImpl**, que deve ficar no mesmo pacote da classe **GenericDAOImpl**. Essa classe deve estender da classe de implementação genérica do DAO e implementar a interface **EstacionamentoDAO**. O exemplo é apresentado no código fonte “*Classe EstacionamentoDAOImpl*”.

```
package br.com.fiap.smartcities.dao.impl;

import br.com.fiap.smartcities.entity.Estacionamento;
import org.springframework.stereotype.Repository;
import br.com.fiap.smartcities.dao.EstacionamentoDAO;

@Repository
public class EstacionamentoDAOImpl extends
GenericDAOImpl<Estacionamento, Integer> implements
EstacionamentoDAO{

}
```

Código Fonte 6.12 – Classe EstacionamentoDAOImpl
Fonte: Elaborado pelo autor (2018)

Observe que adicionamos uma nova anotação sobre a classe. A anotação **@Repository** é parecida com a **@Controller**, pois define que a classe será um componente do *Spring Framework*. A diferença são as responsabilidades desses componentes. Enquanto o controlador possui a responsabilidade de receber a requisição, interagir com o *model* e redirecionar para a *view* correta, o **repository** tem o objetivo de ser uma classe que obtém os dados da aplicação, que podem vir de um banco de dados, arquivos, *web services* e etc., ou seja, qualquer fonte de dados.

Existem 4 tipos principais de componentes do *Spring*, cada um com seus papéis:

- **@Controller:** componente de controle da arquitetura mvc;
- **@Repository:** componente de acesso aos dados;
- **@Service:** componente de serviço, para a camada de negócio;
- **@Component:** componente genérico;

Esses componentes também serão gerenciados pelo *Spring Framework*, dessa forma, não será necessário instanciá-los na aplicação.

A estrutura final da aplicação, até aqui, deve estar conforme a figura “*Camada model da aplicação*”. Temos uma classe *controller*, duas interfaces para o DAOs e duas classes de implementação desses DAOs e uma classe de entidade.

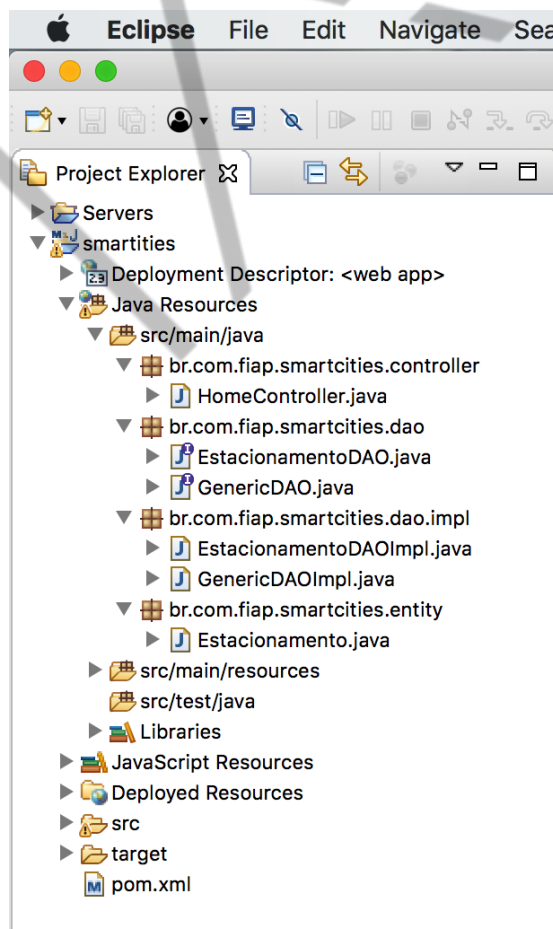


Figura 6.31 – Camada model da aplicação
Fonte: Elaborado pelo autor (2018)

6.5 Adicionando o controller e a view – Cadastro e Listagem

A camada *model* está desenvolvida, agora vamos focar no *controller* e *view*. O primeiro passo é criar a classe que será um *controller*, ou seja, uma classe que vai receber a requisição do usuário, chamar a camada *model* e devolver a *view* correta.

Para isso, clique com o botão direito do mouse no pacote “*br.com.fiap.smartcities.controller*” e escolha a opção “New > Class” (Figura Criando o EstacionamentoController – Parte 1).

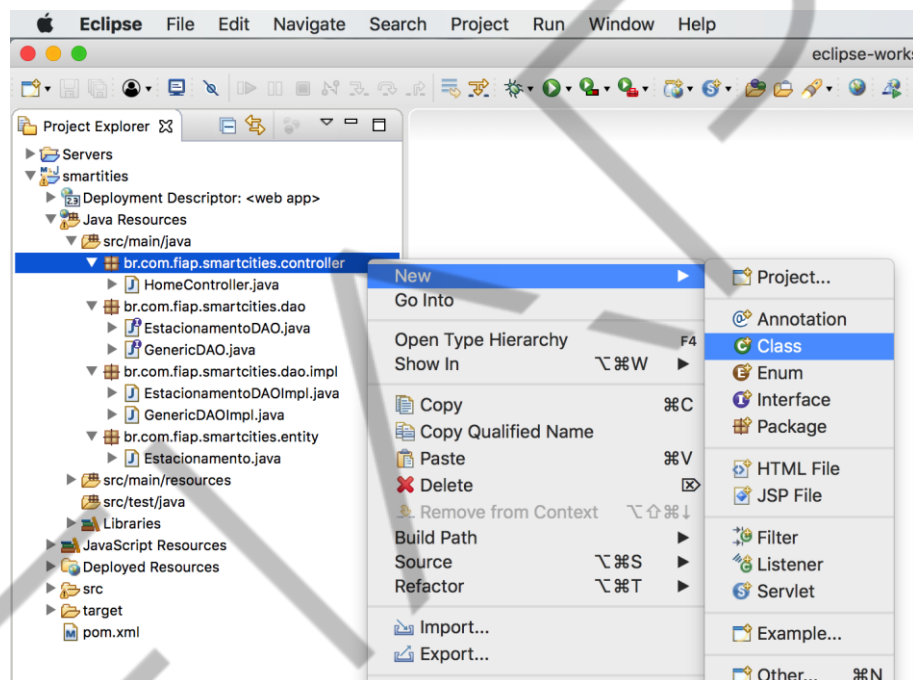


Figura 6.32 – Criando o EstacionamentoController – Parte 1

Fonte: Elaborado pelo autor (2018)

Coloque um nome para a classe, sugiro “*EstacionamentoController*”, conforme a figura “*Criando o EstacionamentoController – Parte 2*”.

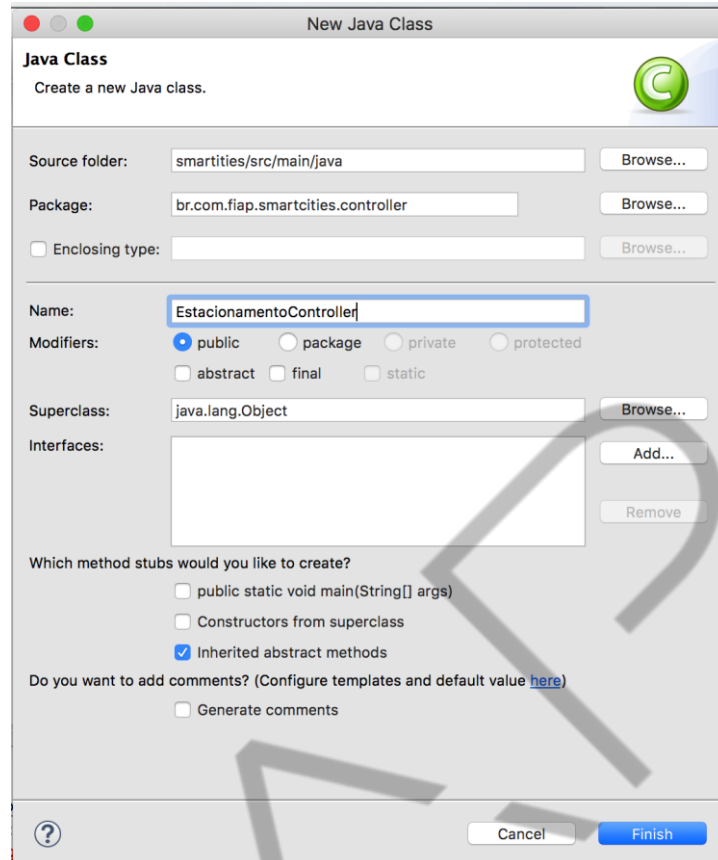


Figura 6.33 – Criando o EstacionamentoController – Parte 2
Fonte: Elaborado pelo autor (2018)

Para transformar uma classe em um componente do *Spring Framework*, vamos utilizar a anotação **@Controller** sobre a classe. Outra configuração necessária é o endereço (URL) que será utilizado para as requisições acessarem esse controlador, assim, adicione a anotação **@RequestMapping** com o valor “estacionamento”, que será parte da URL final.

A primeira funcionalidade implementada será o cadastro. Para isso, desenvolva um método que será responsável por “abrir” a página que possui o formulário de cadastro com os campos do estacionamento. Esse método será mapeado para atender as requisições do tipo GET do HTTP, deve receber um estacionamento como parâmetro (o *framework* se responsabilizará por instanciar e preencher esse parâmetro) e retornar uma *string*, que contém o diretório e o nome da *view*, conforme o código 6.13.

```
@Controller
@RequestMapping("estacionamento")
public class EstacionamentoController {
```

```

    @GetMapping("cadastrar")
    public String abrirForm(Estacionamento
estacionamento) {
        return "estacionamento/cadastro";
    }
}

```

Código Fonte 6.13 – Classe EstacionamentoController
Fonte: Elaborado pelo autor (2018)

Para organizar a aplicação, é recomendado que cada *controller* possua um diretório na pasta de *views*, dessa forma, é possível separar as páginas de cada classe controladora. Clique com o botão direito do mouse na pasta “jsp” e escolha a opção “New > Folder” (Figura Criando a pasta para as páginas do estacionamento – Parte 1).

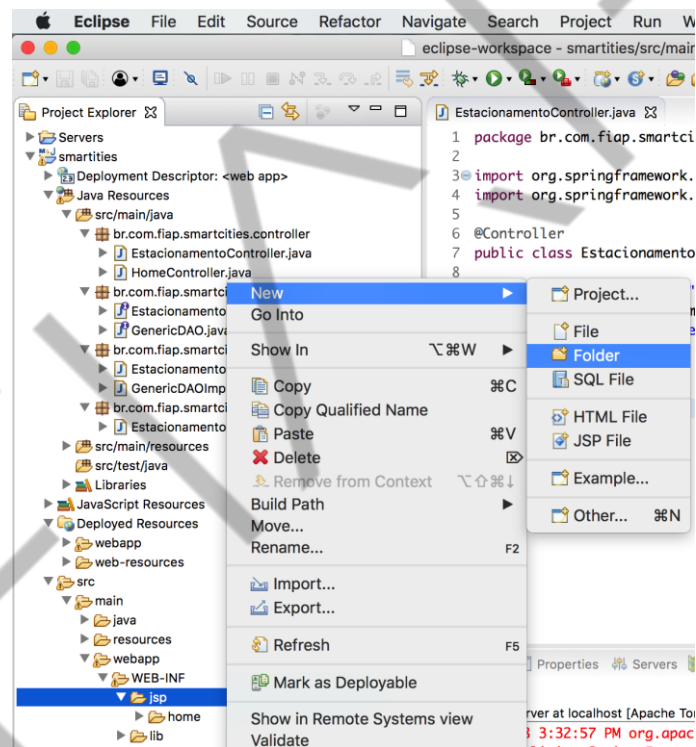


Figura 6.34 – Criando a pasta para as páginas do estacionamento – Parte 1
Fonte: Elaborado pelo autor (2018)

Configure o nome como estacionamento, para ficar igual ao nome do *controller* (Figura Criando a pasta para as páginas do estacionamento – Parte 2).

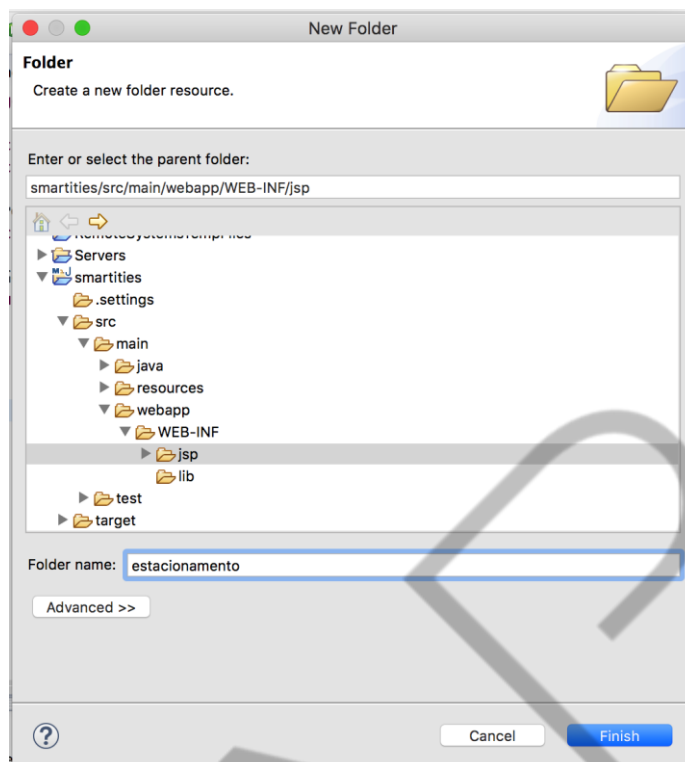


Figura 6.35 – Criando a pasta para as páginas do estacionamento – Parte 2
Fonte: Elaborado pelo autor (2018)

O método “*abrirForm*” do “*EstacionamentoController*” retorna a *String* “*estacionamento/cadastro*”, ou seja, a pasta e o nome da *view*. Acabamos de criar o diretório, agora é o momento de criar a *view*. A nossa *view* será do tipo JSP, assim, crie um arquivo JSP (*New > JSP File*), conforme a figura “Criando a página de cadastro de estacionamento – Parte 1”.

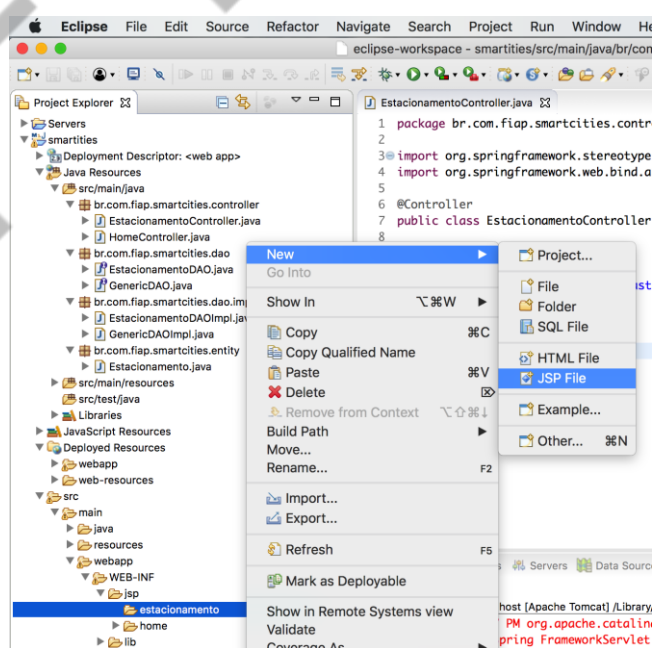


Figura 6.36 – Criando a página de cadastro de estacionamento – Parte 1
Fonte: Elaborado pelo autor (2018)

Configure o nome do arquivo igual ao retorno do método (Figura Criando a página de cadastro de estacionamento – Parte 2), caso contrário, o sistema não vai encontrar a página que deve ser exibida para o usuário.

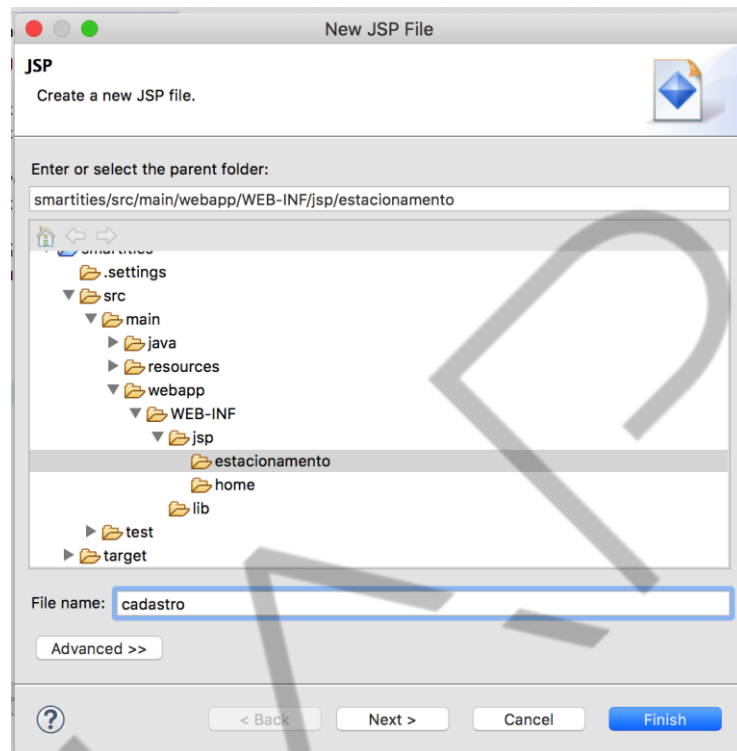


Figura 6.37 – Criando a página de cadastro de estacionamento – Parte 2
Fonte: Elaborado pelo autor (2018)

Para implementar a página de cadastro, utilize as *tags libraries* do JSTL e do *Spring Framework*, pois vão facilitar o nosso desenvolvimento.

Para isso, adicione os comandos:

```
<%@ taglib prefix="form" uri="http://www.springframework.org/tags/form" %>
```

```
<%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core"%>
```

A biblioteca de *tags* core do JSTL possibilita inserir código Java em forma de *tags*. Já a biblioteca de *tags* de *form* do *Spring MVC* vai ajudar no desenvolvimento do formulário de cadastro.

Como pode observar no código fonte Página JSP com o formulário de cadastro de estacionamento, a implementação da página é padrão JSP, com *tags* HTML e algumas *tags* das bibliotecas. A *expression language* **`${msg}`** será utilizada para exibir as mensagens de sucesso e erro para o usuário.

```

<%@ page language="java" contentType="text/html;
charset=UTF-8" pageEncoding="UTF-8"%>
<%@ taglib prefix="form"
uri="http://www.springframework.org/tags/form" %>
<%@ taglib prefix="c"
uri="http://java.sun.com/jsp/jstl/core"%>
<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01
Transitional//EN" "http://www.w3.org/TR/html4/loose.dtd">
<html>
<head>
<meta http-equiv="Content-Type" content="text/html;
charset=UTF-8">
<title>Cadastro de Estacionamento</title>
</head>
<body>
<h1>Cadastro de estacionamento</h1>
${msg }
<c:url value="/estacionamento/cadastrar"
var="action" />
<form:form action="${action}" method="post"
commandName="estacionamento">
<div class="form-group">
<form:label path="nome">Nome</form:label>
<form:input path="nome"/>
</div>
<div class="form-group">
<form:label
path="endereco">Endereço</form:label>
<form:input path="endereco"/>
</div>
<div class="form-group">
<form:label path="vagas">Número de
Vagas</form:label>
<form:input path="vagas" />
</div>
<div class="form-group">
<form:label
path="preco">Preço</form:label>
<form:input path="preco"/>
</div>
<input type="submit" value="Salvar">
</form:form>
</body>
</html>

```

Código Fonte 6.14 – Página JSP com o formulário de cadastro de estacionamento
Fonte: Elaborado pelo autor (2018)

Para configurar a **action** do formulário, utilizamos a tag **<c:url>** para determinar o caminho completo (URL) para o *controller*. Assim, dentro do *controller* será preciso

implementar um método que aceite as requisições do tipo POST na URL “estacionamento/cadastrar”.

As *tags* do *spring* possuem o prefixo `<form:xxx>`. Essa biblioteca possui as *tags* para criar os formulários, *labels*, campos de texto, seleção, *checkbox* etc. Porém, utilizando essas *tags*, o próprio *framework* irá recuperar e exibir as informações vindas do Java.

A primeira *tag* do *spring* utilizada foi `<form:form>`, essa *tag* implementa um formulário padrão do HTML, ela possui as propriedades *action*, que determina a URL em que serão enviadas as informações (nesse caso, será a URL criada pela *tag* `<c:url>`), o método do HTTP (*get* ou *post*) e um *commandName*, que é o nome do parâmetro que possui todos os atributos que serão utilizados no formulário. Lembra do parâmetro do método *abrirForm()* do *controller*? É esse mesmo.

Agora podemos testar! Selecione o projeto e clique com o botão direito do mouse e escolha “Run as > Run on Server”, conforme a figura Teste da funcionalidade de cadastro de estacionamento – Parte 1.

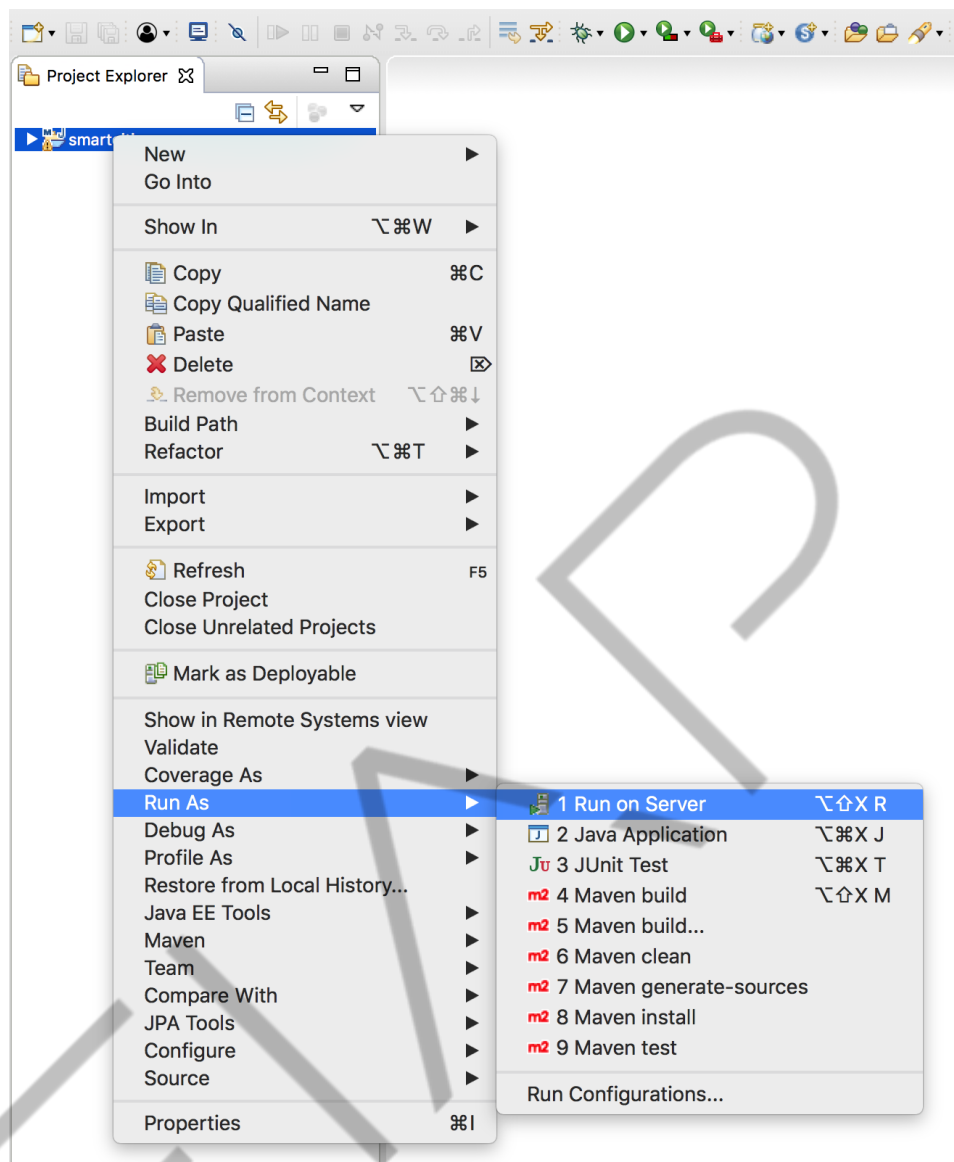


Figura 6.38 – Teste da funcionalidade de cadastro de estacionamento – Parte 1
Fonte: Elaborado pelo autor (2018)

Ajuste a URL para “estacionamento/cadastrar”, conforme configurado no *controller*. O resultado deve ser a página de cadastro de estacionamento (Figura Teste da funcionalidade de cadastro de estacionamento – Parte 1).

Cadastro de Estacionamento

Nome

Endereço

Número de Vagas

Preço

Figura 6.39 – Teste da funcionalidade de cadastro de estacionamento – Parte 1
 Fonte: Elaborado pelo autor (2018)

Por enquanto ainda não existe a ação de cadastro, a única implementação realizada foi a abertura da página de cadastro. Para realmente gravar as informações no banco de dados, precisamos implementar um novo método no *controller* para atender às requisições do tipo POST, na URL “*estacionamento/cadastrar*”.

Você já deve ter percebido que teremos dois métodos no *controller* para realizar o cadastro, um o que abre a página do formulário e outro que realiza o cadastro, após o usuário clicar no botão “Salvar”.

Vamos voltar à classe *controller* e implementar o método **processarForm**, conforme o código fonte “Método POST de cadastro de estacionamento”.

```
@Transactional
@PostMapping("cadastrar")
public ModelAndView processarForm(Estacionamento
estacionamento, RedirectAttributes redirect) {
    try {
        dao.cadastrar(estacionamento);
        redirect.addFlashAttribute("msg",
            "Estacionamento cadastrado");
    } catch (Exception e) {
        return new
            ModelAndView("estacionamento/cadastro").addObject("msg",
            e.getMessage());
    }
    return new
        ModelAndView("redirect:/estacionamento/cadastrar");
}
```

Código Fonte 6.15 – Método POST de cadastro de estacionamento
 Fonte: Elaborado pelo autor (2018)

Esse método deve utilizar a anotação **@PostMapping** com o valor “*cadastrar*”, para que possa atender às requisições do tipo POST da url “*estacionamento/cadastrar*”. Outra anotação é a **@Transactional**, que faz com que o *framework* gerencie a transação que é necessária para efetivar as alterações no banco de dados. Essa anotação sempre será utilizada nos métodos que utilizam os métodos do DAO que modificam os valores no banco de dados (cadastro, atualização e remoção).

O método retorna um objeto do tipo **ModelAndView**, que possibilita retornar o nome da *view* e valores para exibir na página. Outro objeto é o **RedirectAttributes**, utilizado para enviar a mensagem para o usuário após um *redirect*.

Mas o que é um *redirect*? Existem duas formas de enviar uma *view* para o usuário. A primeira é o *forward* ou encaminhamento, nesse modo, o mesmo *request* que foi utilizado para acionar o *controller* é utilizado para enviar a *view* de retorno para o usuário. Dessa forma, se o usuário cadastrar um estacionamento e depois atualizar a página, ele vai cadastrar novamente o estacionamento, já que a requisição original era a requisição de cadastro. Já o *redirect* ou redirecionamento, utiliza uma nova requisição para retornar a *view* para o usuário, assim, caso a página seja atualizada a requisição que será enviada novamente para o servidor será a requisição de retorno da página e não a requisição de cadastro.

Porém, como no *redirect* uma nova requisição é criada, precisamos do objeto **RedirectAttributes** para enviar valores para a tela por essa nova requisição.

Vamos olhar novamente o código e observar que o método realiza o cadastro por meio do DAO e depois adiciona uma mensagem por meio do método **addFlashAttributes**, que possui dois parâmetros que são a chave (o mesmo utilizado pela página JSP) e valor (a mensagem). Esse método especial permite que os valores sejam exibidos mesmo após um *redirect*. No final, o método retorna um objeto do tipo **ModelAndView**, com o valor “**redirect:/estacionamento/cadastrar**”, a palavra **redirect** que determina que será realizado um redirecionamento. Quando isso é realizado, o valor será o endereço (URL) de redirecionamento, (“*/estacionamento/cadastrar*”) e não o diretório e o nome da *view*.

Caso aconteça algum erro, o comando **catch** será acionado e deve retornar um objeto *ModelAndView* sem redirecionar, pois, queremos que o formulário volte preenchido com os valores originais, e com a mensagem de erro.

Agora podemos testar! Execute novamente o projeto, ajuste a URL, preencha os dados e torça! (Figura Teste da funcionalidade de cadastro de estacionamento – Parte 2).



Cadastro de Estacionamento

Nome

Endereço

Número de Vagas

Preço

Figura 6.40 – Teste da funcionalidade de cadastro de estacionamento – Parte 2
Fonte: Elaborado pelo autor (2018)

Após o cadastro, uma mensagem de sucesso deve aparecer. É possível também verificar no console do eclipse as *queries* enviadas para o banco de dados Oracle.



Figura 6.41 – Teste da funcionalidade de cadastro de estacionamento – Parte 3
Fonte: Elaborado pelo autor (2018)

Com o cadastro pronto, vamos implementar a funcionalidade de listagem de estacionamentos.

O primeiro passo é adicionar um método na classe *controller* que deve receber a requisição, chamar o DAO para recuperar todos os estacionamentos cadastrados e encaminhar a lista de estacionamentos e a *view* para a resposta para o usuário. Código fonte “Método de listagem do controller”.

```
@GetMapping("listar")
public ModelAndView listar() {
    return new ModelAndView("estacionamento/lista").addObject("estacionamentos", dao.listar());
}
```

Código Fonte 6.16 – Método de listagem do controller
Fonte: Elaborado pelo autor (2018)

O método é acionado pela URL “estacionamento/listar” e devolve a *view* “estacionamento/lista”. Dessa forma, crie a página “lista.jsp” dentro do diretório “estacionamento” da *views*. Figura “Criando a view para listar os estacionamentos – Parte 1”.

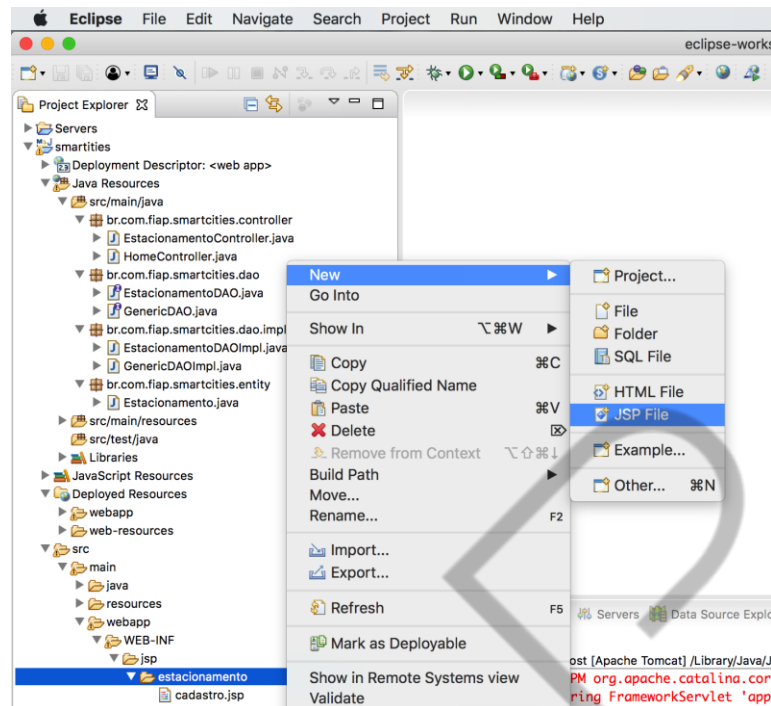


Figura 6.42 – Criando a view para listar os estacionamentos – Parte 1
Fonte: Elaborado pelo autor (2018)

O nome da página deve ser igual ao configurado do retorno do método listar (lista), conforme a figura “*Criando a view para listar os estacionamentos – Parte 2*”.

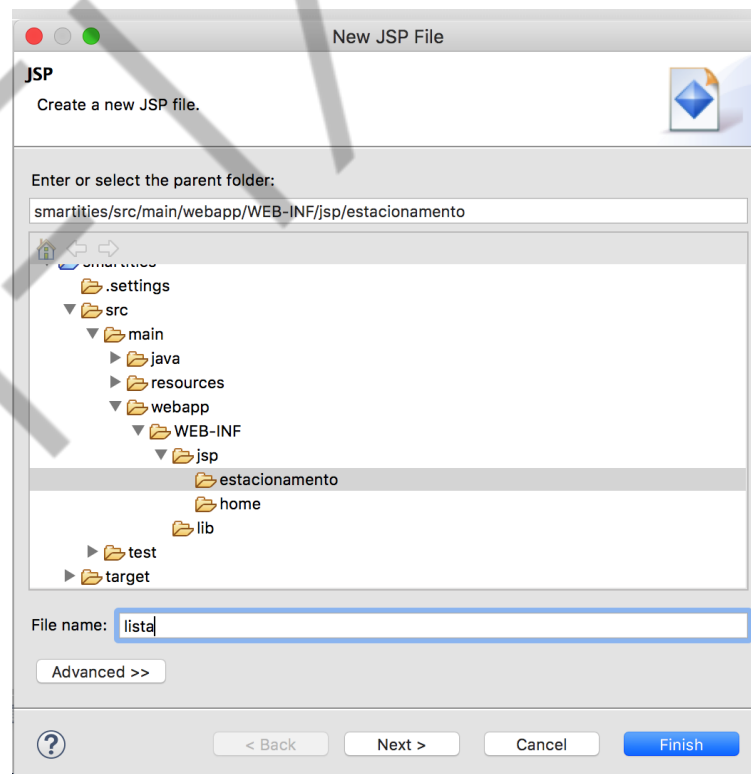


Figura 6.43 – Criando a view para listar os estacionamentos – Parte 2
Fonte: Elaborado pelo autor (2018).

Utilize as *tags* de JSTL para criar uma tabela HTML para exibir os dados dos estacionamentos (Código fonte Página JSP de listagem de estacionamentos).

```
<%@ page language="java" contentType="text/html;
charset=UTF-8" pageEncoding="UTF-8"%>
<%@ taglib prefix="c"
uri="http://java.sun.com/jsp/jstl/core"%>
<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01
Transitional//EN" "http://www.w3.org/TR/html4/loose.dtd">
<html>
<head>
<meta http-equiv="Content-Type" content="text/html;
charset=UTF-8">
<title>Lista de Estacionamento</title>
</head>
<body>
<h1>Estacionamentos cadastrados</h1>
<table class="table">
<tr>
<th>Nome</th>
<th>Endereço</th>
<th>Vagas</th>
<th>Preço</th>
</tr>
<c:forEach items="${estacionamentos}" var="e">
<tr>
<td>${e.nome}</td>
<td>${e.endereco}</td>
<td>${e.vagas}</td>
<td>${e.preco}</td>
</tr>
</c:forEach>
</table>
</body>
</html>
```

Código Fonte 6.17 – Página JSP de listagem de estacionamentos

Fonte: Elaborado pelo autor (2018)

A tag `<c:forEach>` recebe a lista de estacionamentos enviado pelo *controller* e cria cada linha da tabela HTML.

Agora podemos testar! Execute o projeto e ajuste a URL para “*estacionamento/listar*” (Figura 6.44). Observe que é possível ver o *select* no console do eclipse.

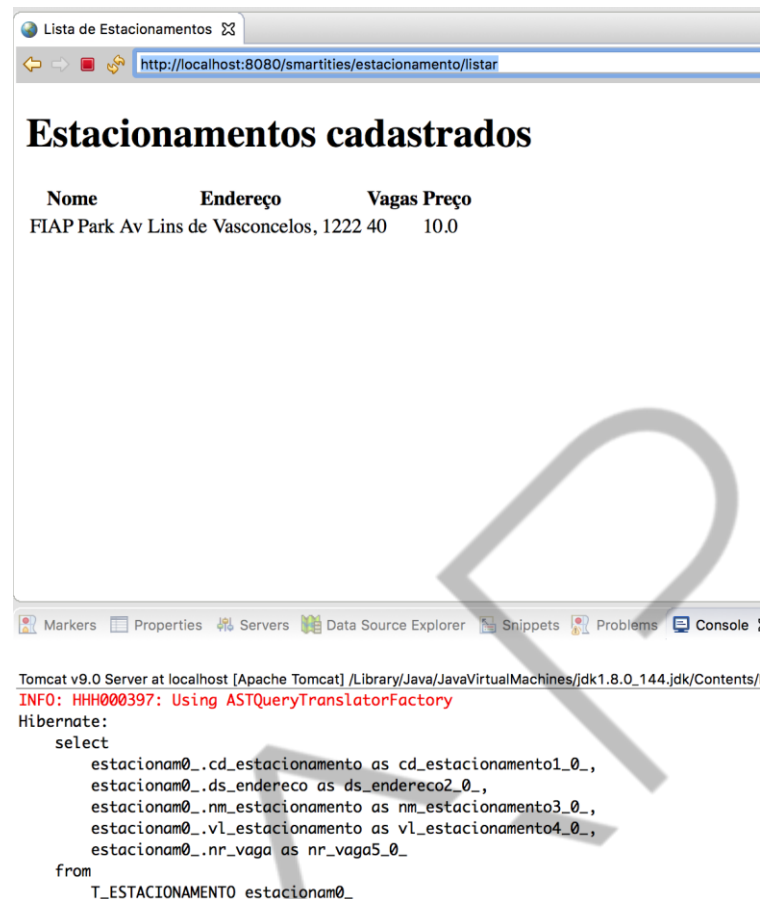


Figura 6.44 – Teste da funcionalidade de listar estacionamentos
Fonte: Elaborado pelo autor (2018)

6.6 Layout

A nossa aplicação está funcional, porém o *layout* está muito simples. Precisamos melhorá-la! Para isso, vamos utilizar o *bootstrap*, um *framework* *css* e *js* muito utilizado no mercado atualmente.

Faça o *download* dos arquivos pelo link <https://getbootstrap.com/>, aproveite para dar uma olhada na documentação! É muito interessante e fácil de entender.

Depois de realizar o *download*, copie os arquivos para dentro do projeto. Crie um diretório chamado “resources” dentro dos diretórios “src/main/webapp/” e cole as pastas “css” e “js” do *bootstrap* (Figura - Arquivos do *bootstrap*). Observe também que é preciso adicionar a biblioteca do *jQuery*, pois o *bootstrap* depende dela. Faça o *download* do *jQuery* pelo link <https://jquery.com/> e coloque o arquivo na pasta *js*.

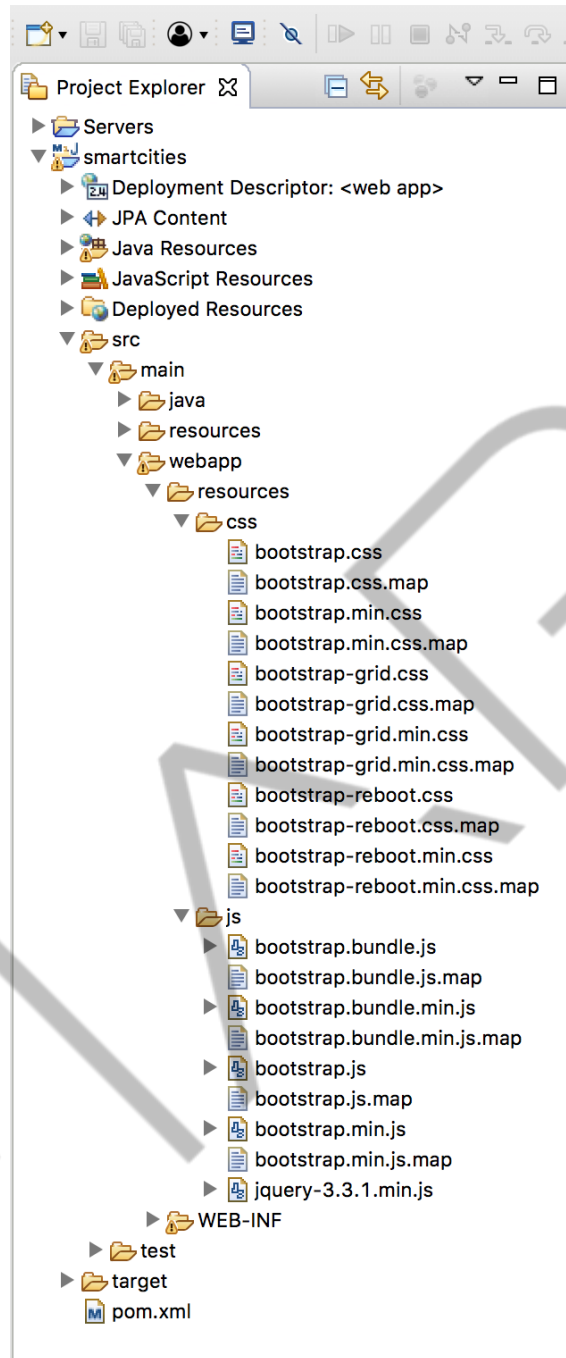


Figura 6.45 – Arquivos do bootstrap
Fonte: Elaborado pelo autor (2018)

Para que todas as páginas possuam o mesmo *layout*, vamos criar um *template*. Ele deve possuir todas as partes comuns de todas as páginas, como a barra de navegação, rodapé etc.

Esse *template* será uma *tag* JSP, crie um diretório dentro da pasta “WEB-INF” chamada “tags” (Figura - Pasta para o template).

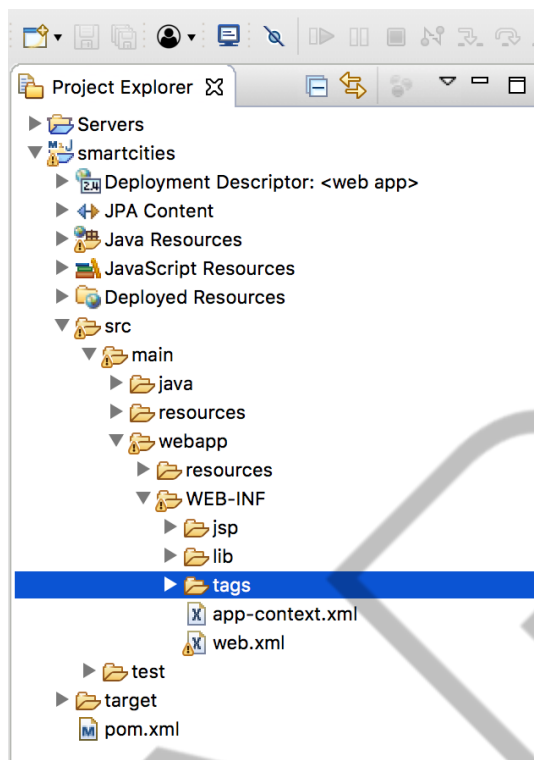


Figura 6.46 – Pasta para o template
Fonte: Elaborado pelo autor (2018)

Nessa pasta, clique com o botão direito do mouse e escolha a opção “New > Other”, conforme a figura “Criando uma nova tag JSP - Parte 1”.

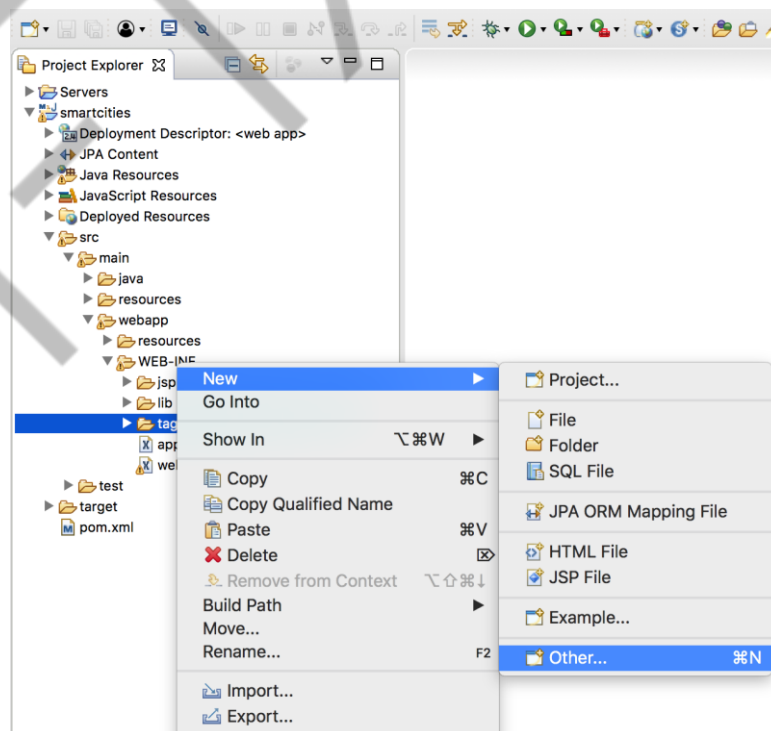


Figura 6.47 – Criando uma nova tag JSP - Parte 1
Fonte: Elaborado pelo autor (2018)

Procure por “JSP Tag”(Figura - Criando uma nova tag JSP - Parte 2) e clique em “Next”.

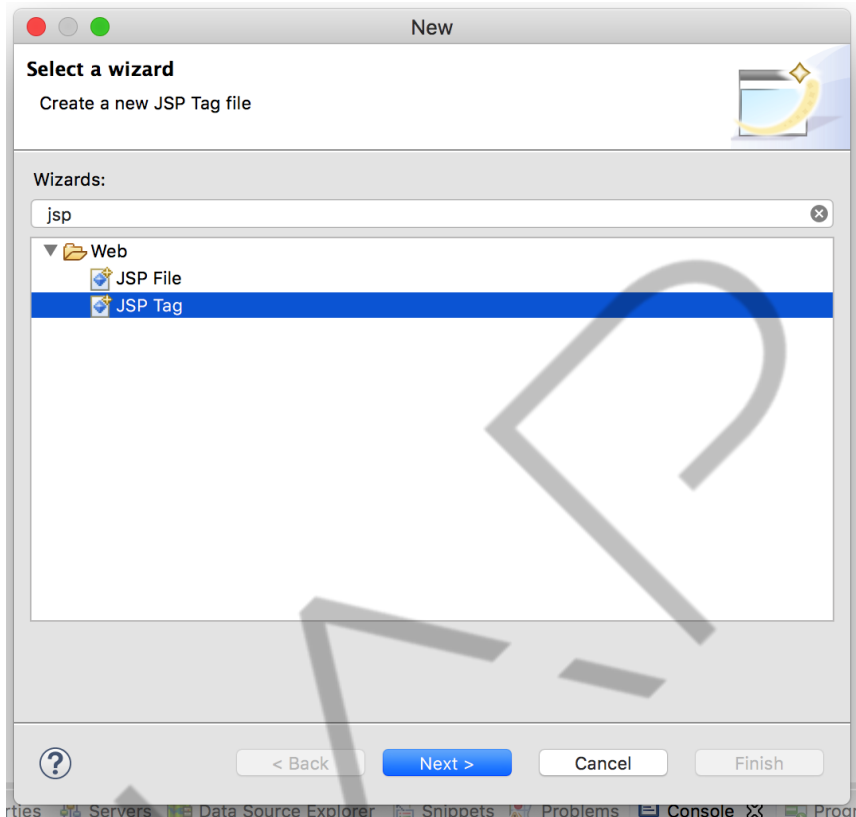


Figura 6.48 – Criando uma nova tag JSP - Parte 2

Fonte: Elaborado pelo autor (2018)

Vamos dar o nome de “*template*” para a *tag*. Esse nome será utilizado pelas páginas JSPs (Figura - Criando uma nova tag JSP - Parte 3).

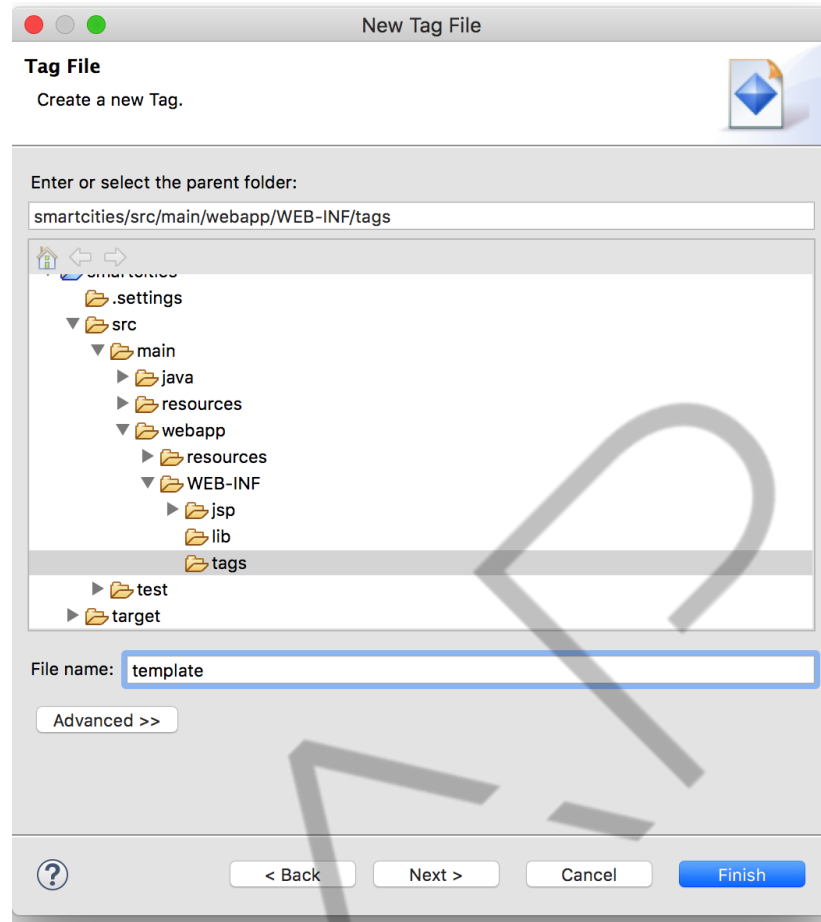


Figura 6.49 – Criando uma nova tag JSP - Parte 3
 Fonte: Elaborado pelo autor (2018)

Estamos criando uma *tag* JSP, parecidas como `<c:url>`, `<c:forEach>` e etc. Essa *tag* deve conter toda a estrutura básica do HTML e tudo que é comum a todas as páginas, como os links do *css* e *javascript*, *layout* da página, o rodapé e o menu. E também deve conter uma área para as outras páginas adicionarem seus conteúdos, isso é feito pela tag `<jsp:doBody>`, conforme o código fonte *Template* para as páginas do sistema.

```
<%@ tag language="java" pageEncoding="UTF-8"%>
<%@
                                taglib                                prefix="c"
uri="http://java.sun.com/jsp/jstl/core"%>
    <%@ attribute name="title" required="true" %>
    <!DOCTYPE html>
    <html>
    <head>
    <meta    http-equiv="Content-Type"    content="text/html;
charset=UTF-8">
    <title>${title }</title>
    <link rel="stylesheet" type="text/css"
        href="<c:url
value="/resources/css/bootstrap.min.css"/>">
```

```

</head>
<body>
  <nav class="navbar navbar-expand-lg navbar-light bg-
light">
    <a class="navbar-brand" href="#">Smart Cities</a>
    <button class="navbar-toggler" type="button" data-
toggle="collapse" data-target="#navbarSupportedContent" aria-
controls="navbarSupportedContent" aria-expanded="false" aria-
label="Toggle navigation">
      <span class="navbar-toggler-icon"></span>
    </button>

    <div class="collapse navbar-collapse"
id="navbarSupportedContent">
      <ul class="navbar-nav mr-auto">
        <li class="nav-item active">
          <a class="nav-link" href='<c:url
value="/" />'>Home</a>
        </li>
        <li class="nav-item dropdown">
          <a class="nav-link dropdown-toggle" href="#"
id="navbarDropdown" role="button" data-toggle="dropdown" aria-
haspopup="true" aria-expanded="false">
            Estacionamento
          </a>
          <div class="dropdown-menu" aria-
labelledby="navbarDropdown">
            <a class="dropdown-item" href='<c:url
value="/estacionamento/cadastrar/">'>Cadastrar</a>
            <a class="dropdown-item" href='<c:url
value="/estacionamento/listar/">'>Listar</a>
          </div>
        </li>
      </ul>
    </div>
  </nav>
  <div class="container">
    <jsp:doBody />
  </div>
  <script src="<c:url value="/resources/js/jquery-
3.3.1.min.js"/>"></script>
  <script src="<c:url
value="/resources/js/bootstrap.min.js"/>"></script>
</body>
</html>

```

Código Fonte 6.18 – Template para as páginas do sistema
Fonte: Elaborado pelo autor (2018)

Observe que adicionamos os links do *css* e *javascript* do *bootstrap* e *jquery*. Os *javascript* foram adicionados ao final da página por boas práticas, já que ela é

carregada de cima para baixo. Criamos uma barra de navegação padrão do *bootstrap* com os links para as páginas de cadastro e listagem de estacionamentos. Para funcionar corretamente, os links foram criados utilizando a tag `<c:url>`. Isso foi aplicado também para os arquivos de **js** e **css**.

No *template*, foi declarado um atributo obrigatório **title**, que deve ser definido pelas páginas que utilizarem o *template* para que o título de cada página seja diferente.

Com o *template* pronto, vamos ajustar as páginas. Abra a página *index.jsp* que está dentro do diretório *jsp/home*. Para utilizar o *template*, basta declarar a *taglib* que foi criada: `<%@ taglib prefix="tags" tagdir="/WEB-INF/tags" %>`. A diferença é que devemos informar no parâmetro **tagdir** o diretório onde a *tag* se encontra. Depois, utilizamos a *tag* de *template* `<tags:template>` definindo o título da página (Código - Fonte Página inicial utilizando o *template*). Dessa forma, a página inicial recebe toda a estrutura do *layout* e tudo que estiver dentro da *tag* de *template* será adicionado no lugar da tag `<jsp:doBody>` do *template*.

```
<%@ page language="java" contentType="text/html;
charset=UTF-8" pageEncoding="UTF-8"%>

<%@ taglib prefix="form"
uri="http://www.springframework.org/tags/form" %>
<%@ taglib prefix="c"
uri="http://java.sun.com/jsp/jstl/core"%>
<%@ taglib prefix="tags" tagdir="/WEB-INF/tags" %>

<tags:template title="Cadastro de Estacionamento">

    <h1>Olá Mundo!</h1>

</tags:template>
```

Código Fonte 6.19 – Página inicial utilizando o *template*
Fonte: Elaborado pelo autor (2018)

Vamos testar o novo *layout* da aplicação! Repita o processo para executar o projeto, clique com o botão direito do mouse em cima do projeto e escolha a opção “Run As > Run on Server”. Se tudo der certo, uma página deve abrir como na figura - Página inicial da aplicação.



Figura 6.50 – Página inicial da aplicação
Fonte: Elaborado pelo autor (2018)

Os *links* podem ser utilizados! Porém, precisamos ajustar as outras páginas para que elas utilizem o *template* também.

Ajuste a página de cadastro, como no código Página de cadastro utilizando o *template*.

```
<%@ page language="java" contentType="text/html;
charset=UTF-8" pageEncoding="UTF-8"%>

<%@ taglib prefix="form"
uri="http://www.springframework.org/tags/form" %>
<%@ taglib prefix="c"
uri="http://java.sun.com/jsp/jstl/core"%>
<%@ taglib prefix="tags" tagdir="/WEB-INF/tags" %>

<tags:template title="Cadastro de Estacionamento">

    <h1>Cadastro de estacionamento</h1>
    ${msg }
    <c:url value="/estacionamento/cadastrar"
var="action"/>
    <form:form action="${action }" method="post"
commandName="estacionamento">
        <div class="form-group">
            <form:label path="nome">Nome</form:label>
            <form:input path="nome" cssClass="form-
control"/>
        </div>
        <div class="form-group">
```

```

        <form:label
path="endereco">Endereço</form:label>
        <form:input                                path="endereco"
cssClass="form-control"/>
    </div>
    <div class="form-group">
        <form:label            path="vagas">Número        de
Vagas</form:label>
        <form:input path="vagas" cssClass="form-
control"/>
    </div>
    <div class="form-group">
        <form:label
path="preco">Preço</form:label>
        <form:input path="preco" cssClass="form-
control"/>
    </div>
    <input type="submit" value="Salvar" class="btn
btn-primary">
    </form:form>

</tags:template>

```

Código Fonte 6.20 – Página de cadastro utilizando o template
Fonte: Elaborado pelo autor (2018)

Agora podemos adicionar as classes do *bootstrap* para dar o estilo aos campos do formulário. As *tags* de formulário do *Spring* possuem o atributo **cssClass**, que adiciona uma classe na *tag* HTML. O resultado pode ser visto na figura Página de cadastro com o *template*.

Cadastro de Estacionamento

Nome

Endereço

Número de Vagas

Preço

Salvar

Figura 6.51 – Página de cadastro com o template
Fonte: Elaborado pelo autor (2018)

O último ajuste é na página de listagem, utilize o *template* e adicione as classes css do *bootstrap* para estilizar a tabela (Figura - Página de listagem utilizando o *template*).

```
<%@ page language="java" contentType="text/html;
charset=UTF-8" pageEncoding="UTF-8"%>

<%@ taglib prefix="form"
uri="http://www.springframework.org/tags/form" %>
<%@ taglib prefix="c"
uri="http://java.sun.com/jsp/jstl/core"%>
<%@ taglib prefix="tags" tagdir="/WEB-INF/tags" %>

<tags:template title="Lista de Estacionamento">

    <h1>Estacionamentos cadastrados</h1>
    ${msg }
    <table class="table">
        <tr>
            <th>Nome</th>
            <th>Endereço</th>
            <th>Vagas</th>
            <th>Preço</th>
        </tr>
        <c:forEach items="${estacionamentos }" var="e">
            <tr>
                <td>${e.nome }</td>
                <td>${e.endereco }</td>
                <td>${e.vagas }</td>
                <td>${e.preco }</td>
            </tr>
        </c:forEach>
    </table>

</tags:template>
```

Código Fonte 6.21 – Página de listagem utilizando o template
Fonte: Elaborado pelo autor (2018)

Pronto! Nossas páginas estão padronizadas e muito mais elegantes! (Figura - Página de listagem com o *template*).



Figura 6.52 – Página de listagem com o template
Fonte: Elaborado pelo autor (2018)

6.7 Finalizando o CRUD! Atualizar e Excluir

Vamos implementar as duas últimas funcionalidades: atualizar e remover os estacionamentos.

Para atualizar um estacionamento, o primeiro passo é permitir que o usuário escolha o estacionamento que será modificado. Para isso, adicione na listagem um botão de edição ao lado de cada linha da tabela de listagem de estacionamento (Figura - Botão de edição na listagem de estacionamentos).



Figura 6.53 – Botão de edição na listagem de estacionamentos
Fonte: Elaborado pelo autor (2018)

Na página de listagem, adicione uma coluna de título e outra de conteúdo com um botão que é um *link* que envia o código do estacionamento e aciona o *EstacionamentoController*, conforme o código fonte Página de listagem com o botão de edição.

```
<%@ page language="java" contentType="text/html;
charset=UTF-8" pageEncoding="UTF-8"%>
```

```

<%@           taglib           prefix="form"
uri="http://www.springframework.org/tags/form" %>
<%@           taglib           prefix="c"
uri="http://java.sun.com/jsp/jstl/core"%>
<%@ taglib prefix="tags" tagdir="/WEB-INF/tags" %>

<tags:template title="Lista de Estacionamento">

    <h1>Estacionamentos cadastrados</h1>
    ${msg }
    <table class="table">
        <tr>
            <th>Nome</th>
            <th>Endereço</th>
            <th>Vagas</th>
            <th>Preço</th>
            <th></th>
        </tr>
        <c:forEach items="${estacionamentos }" var="e">
            <tr>
                <td>${e.nome }</td>
                <td>${e.endereco }</td>
                <td>${e.vagas }</td>
                <td>${e.preco }</td>
                <td>
                    <c:url
value="/estacionamento/editar/${e.codigo }" var="link"/>
                    <a href="${link}" class="btn
btn-primary btn-sm">Editar</a>
                </td>
            </tr>
        </c:forEach>
    </table>
</tags:template>

```

Código Fonte 6.22 – Página de listagem com o botão de edição

Fonte: Elaborado pelo autor (2018)

Observe que o link foi criado utilizando a tag `<c:url>` com o código do estacionamento ao final da URL. Agora precisamos criar um método na classe **EstacionamentoController** que atenda à url `"editar/{id}"` com o método GET do HTTP (Código fonte - Método para abrir a tela de edição).

```

@GetMapping("editar/{id}")
public ModelAndView editar(@PathVariable("id") int
codigo) {
    return new
ModelAndView("estacionamento/edicao").addObject("estacionamen
to", dao.buscar(codigo));

```

```
}
```

Código Fonte 6.23 – Método para abrir a tela de edição
Fonte: Elaborado pelo autor (2018)

Esse método deve receber o **id** do estacionamento que será editado para buscar no banco de dados as informações do estacionamento e abrir a tela com o formulário preenchido com esses dados.

O método retorna um objeto do tipo **ModelAndView**, para informar ao framework qual é a *view* e os dados que serão exibidos para o usuário. A anotação **@GetMapping** configura a URL com o valor “*editar/{id}*”, o {id} é um valor enviado pela URL, nesse caso o **id** do estacionamento. Esse valor deve ser inserido no parâmetro código do método, por isso, anotamos esse parâmetro com **@PathVariable(“id”)**, dessa forma, o *framework* recupera o valor da URL e coloca no parâmetro do método.

O método basicamente retorna um novo objeto do tipo *ModelAndView*, informando a *view* que será aberta (estacionamento/edição) e os dados do estacionamento, que foram recuperados do banco de dados por meio do DAO.

O próximo passo é construir a página de edição que deve conter um formulário com os dados do estacionamento. É uma página muito parecida com a do cadastro, a diferença é na *action* do formulário, que deve ter a URL de edição e o formulário, que deve possuir também um campo oculto com o código do estacionamento (Código fonte Tela para atualizar um estacionamento).

```
<%@ page language="java" contentType="text/html;
charset=UTF-8" pageEncoding="UTF-8"%>

<%@ taglib prefix="form"
uri="http://www.springframework.org/tags/form" %>
<%@ taglib prefix="c"
uri="http://java.sun.com/jsp/jstl/core"%>
<%@ taglib prefix="tags" tagdir="/WEB-INF/tags" %>

<tags:template title="Edição de Estacionamento">

    <h1>Edição de estacionamento</h1>
    ${msg }
    <c:url value="/estacionamento/editar" var="action"/>
    <form:form action="${action}" method="post"
commandName="estacionamento">
        <form:hidden path="codigo"/>
        <div class="form-group">
```

```

        <form:label path="nome">Nome</form:label>
        <form:input path="nome" cssClass="form-
control"/>
    </div>
    <div class="form-group">
        <form:label
path="endereco">Endereço</form:label>
        <form:input path="endereco"
cssClass="form-control"/>
    </div>
    <div class="form-group">
        <form:label path="vagas">Número de
Vagas</form:label>
        <form:input path="vagas" cssClass="form-
control"/>
    </div>
    <div class="form-group">
        <form:label
path="preco">Preço</form:label>
        <form:input path="preco" cssClass="form-
control"/>
    </div>
    <input type="submit" value="Salvar" class="btn
btn-primary">
    <c:url value="/estacionamento/listar"
var="link"/>
    <a href="${link}" class="btn btn-
danger">Cancelar</a>
</form:form>

</tags:template>

```

Código Fonte 6.24 – Tela para atualizar um estacionamento

Fonte: Elaborado pelo autor (2018)

Para finalizar, é preciso desenvolver o método no *controller* que irá receber as informações do estacionamento e chamar o DAO para atualizar o banco de dados, conforme o código “Método para atualizar um estacionamento”.

```

@Transactional
@PostMapping("editar")
public ModelAndView editar(Estacionamento estacionamento,
RedirectAttributes redirect) {
    try {
        dao.atualizar(estacionamento);
        redirect.addFlashAttribute("msg",
"Atualizado");
    } catch (Exception e) {
        return new
ModelAndView("estacionamento/edicao").addObject("msg",
e.getMessage());
    }
}

```

```
    }  
    return new  
    ModelAndView("redirect:/estacionamento/listar");  
}
```

Código Fonte 6.25 – Método para atualizar um estacionamento
Fonte: Elaborado pelo autor (2018)

O método atende à requisição do tipo POST e recebe os dados do estacionamento e o objeto do tipo *RedirectAttributes*, para ser possível enviar informações após um *Redirect*.

O DAO é utilizado para atualizar os dados no banco e, se tudo der certo, uma mensagem de sucesso é adicionada no *RedirectAttributes* e depois o usuário é redirecionado para a listagem. Lembre-se sempre de realizar um *redirect* após um POST.

Se algum erro acontecer, o usuário é encaminhado para a página de edição com uma mensagem de erro. O resultado pode ser visto nas figuras “Tela de atualização de estacionamento” e “Mensagem de sucesso após a atualização”.

A imagem mostra uma interface web no navegador com o título 'Edição de estacionamento'. No topo, há uma barra de navegação com 'Smart Cities', 'Home' e 'Estacionamento'. O formulário principal contém campos para: Nome (FIAP Park), Endereço (Av Lins de Vasconcelos, 1222), Número de Vagas (40) e Preço (10.0). Abaixo dos campos, há dois botões: 'Salvar' (azul) e 'Cancelar' (vermelho).

Figura 6.54 – Tela de atualização de estacionamento
Fonte: Elaborado pelo autor (2018)

Após modificar e enviar os dados, o usuário é redirecionado para a página de listagem com a mensagem de sucesso.

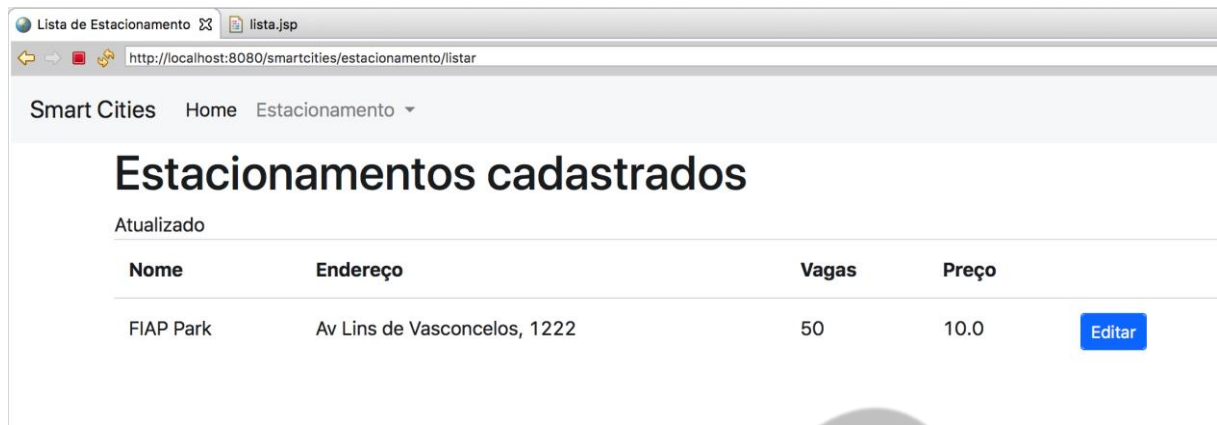


Figura 6.55 – Mensagem de sucesso após a atualização
Fonte: Elaborado pelo autor (2018)

A última funcionalidade será a remoção de um estacionamento. Para isso, vamos adicionar um botão na tela de listagem, ao lado do botão de edição.

Esse botão deve abrir um modal para confirmar a exclusão pelo usuário (Código fonte Modal e botão para a exclusão de um estacionamento).

```
<%@ page language="java" contentType="text/html;
charset=UTF-8" pageEncoding="UTF-8"%>

<%@ taglib prefix="form"
uri="http://www.springframework.org/tags/form" %>
<%@ taglib prefix="c"
uri="http://java.sun.com/jsp/jstl/core"%>
<%@ taglib prefix="tags" tagdir="/WEB-INF/tags" %>

<tags:template title="Lista de Estacionamento">

    <h1>Estacionamentos cadastrados</h1>
    ${msg }
    <table class="table">
        <tr>
            <th>Nome</th>
            <th>Endereço</th>
            <th>Vagas</th>
            <th>Preço</th>
            <th></th>
        </tr>
        <c:forEach items="${estacionamentos }" var="e">
            <tr>
                <td>${e.nome }</td>
                <td>${e.endereco }</td>
                <td>${e.vagas }</td>
                <td>${e.preco }</td>
                <td>
                    <c:url
value="/estacionamento/editar/${e.codigo }" var="link"/>
```

```

<a href="${link}" class="btn
btn-primary btn-sm">Editar</a>
        <button type="button"
onclick="codigo.value = ${e.codigo}" class="btn btn-danger
btn-sm" data-toggle="modal" data-target="#excluirModal">
            Excluir
        </button>
    </td>
</tr>
</c:forEach>
</table>

<!-- Modal -->
<div class="modal fade" id="excluirModal"
tabindex="-1" role="dialog" aria-
labelledby="exampleModalLabel" aria-hidden="true">
    <div class="modal-dialog" role="document">
        <div class="modal-content">
            <div class="modal-header">
                <h5 class="modal-title"
id="exampleModalLabel">Confirmação</h5>
                <button type="button" class="close" data-
dismiss="modal" aria-label="Close">
                    <span aria-hidden="true">&times;</span>
                </button>
            </div>
            <div class="modal-body">
                Realmente deseja excluir o estacionamento?
            </div>
            <div class="modal-footer">
                <c:url value="/estacionamento/excluir"
var="action"/>
                <form action="${action}" method="post">
                    <input type="hidden" name="codigo"
id="codigo">
                    <button type="button" class="btn btn-
secondary" data-dismiss="modal">Cancelar</button>
                    <button type="submit" class="btn btn-
danger">Excluir</button>
                </form>
            </div>
        </div>
    </div>
</div>
</div>
</div>
</tags:template>

```

Código Fonte 6.26 – Modal e botão para a exclusão de um estacionamento.

Fonte: Elaborado pelo autor (2018)

O botão basicamente abre o modal e passa o **id** do estacionamento para o campo oculto que está no formulário dentro do modal (Figura Botão de exclusão na tela de listagem).

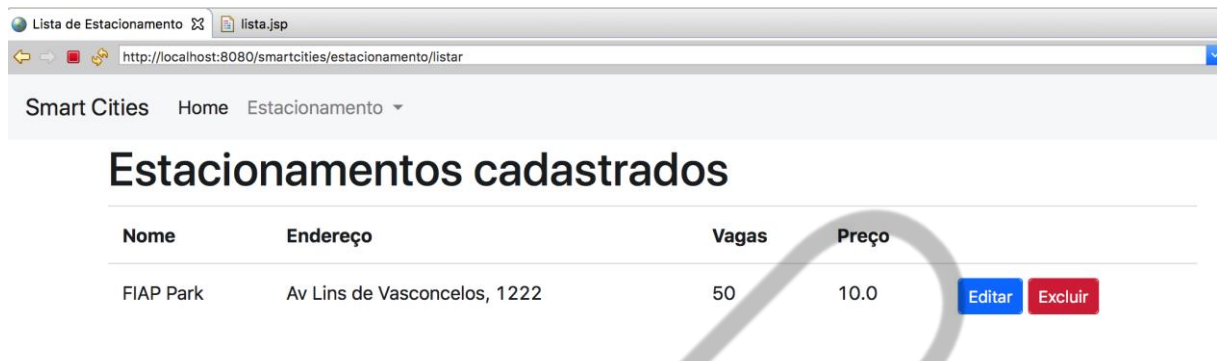


Figura 6.56 – Botão de exclusão na tela de listagem.
Fonte: Elaborado pelo autor (2018)

O modal possui um cabeçalho e um corpo. No cabeçalho, temos o título e um botão para fechar o modal. Já no corpo, exibimos uma mensagem para confirmar a exclusão e desenvolvemos um formulário com o código do estacionamento e dois botões, um para enviar o formulário e confirmar a exclusão e outro para cancelar a operação (Figura Modal para confirmação da exclusão do estacionamento).

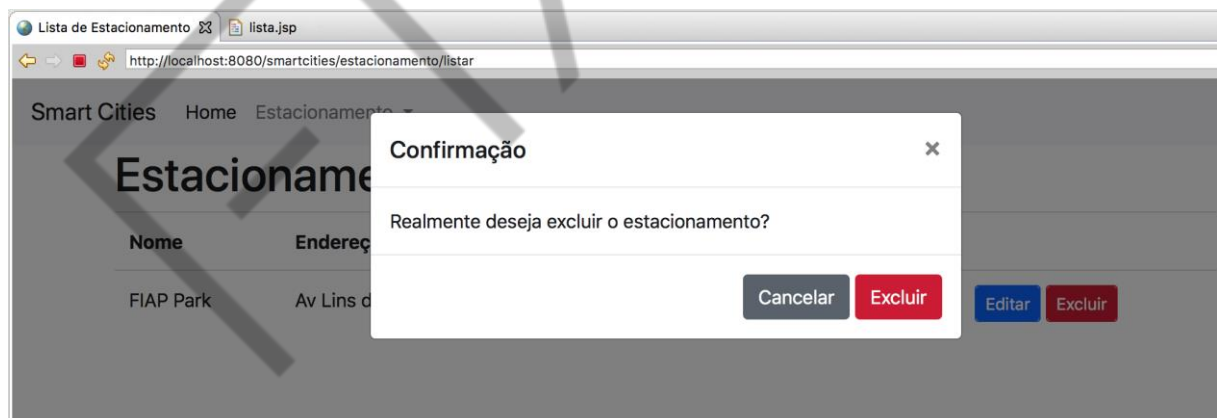


Figura 6.57 – Modal para confirmação da exclusão do estacionamento.
Fonte: Elaborado pelo autor (2018)

Agora é preciso desenvolver o método no *controller* que será executado após o clique do botão excluir do modal. O método vai receber o código do estacionamento e o objeto *RedirectAttributes*, para enviar uma mensagem de sucesso ou erro (Código fonte Método do *controller* para exclusão de um estacionamento.).

```
@Transactional
@PostMapping("excluir")
```

```
public String excluir(int codigo, RedirectAttributes
redirect) {
    try {
        dao.remover(codigo);
        redirect.addFlashAttribute("msg",
"Excluido!");
    } catch (Exception e) {
        redirect.addFlashAttribute("msg",
e.getMessage());
    }
    return "redirect:/estacionamento/listar";
}
```

Código Fonte 6.26 – Método do controller para exclusão de um estacionamento.
Fonte: Elaborado pelo autor (2018)

Esse método também precisa da anotação **@Transactional** para realizar o *commit*. Após a exclusão, o usuário é redirecionado para a tela de listagem com a mensagem de sucesso. Caso aconteça algum erro, o usuário também é redirecionado para a tela de listagem, porém com a mensagem de erro.

A figura abaixo apresenta a mensagem de sucesso após a exclusão de um estacionamento.

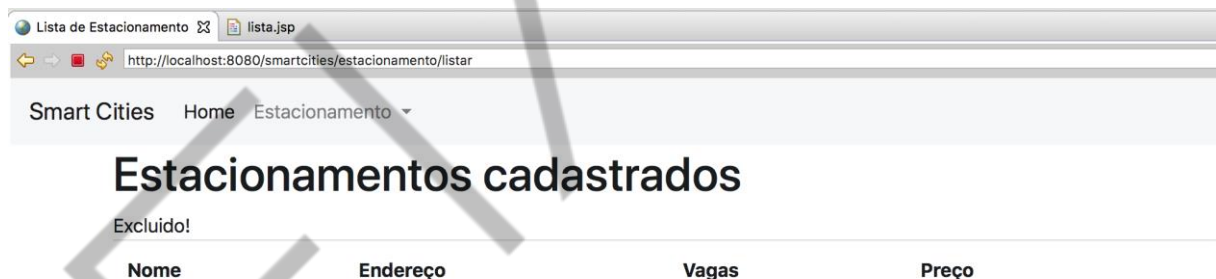


Figura 6.58 – Mensagem de sucesso após a exclusão.
Fonte: Elaborado pelo autor (2018)

Com isso, finalizamos as operações básicas com o *Spring MVC* e *JPA/Hibernate*. Abordamos como configurar o projeto e trabalhar com a estrutura *MVC*, implementando os *controllers* e *repositories*, com injeção de dependências. Vimos também como trabalhar com *templates*, utilizando *tags* JSP e o *framework bootstrap*, para deixar as páginas mais elegantes!

REFERÊNCIAS

LUCKOW, D.H.; Melo, A.A. **Programação Java para Web**. São Paulo: Novatec, 2010.

HEMRAJANI, Anil. **Java com Spring, Hibernate e Eclipse**. São Paulo: Pearson, 2013.

LEMAY, L.; COLBURN, R.; TYLER, D. **Aprenda a criar páginas Web com HTML e XHTML em 21 dias**. 1ª ed. São Paulo: Pearson Education do Brasil, 2002. *

DEITEL, P.J.; DEITEL, H. M. **Ajax Rich Internet Applications e Desenvolvimento Web para Programadores**. 1ª ed. São Paulo: Pearson Education do Brasil, 2008. *